Spatial Analysis and Modeling

Modeling Spatial Dependence

UNIVERSITÀ
DI TORINO

Learning Objectives

- Understand **spatial dependence** theory and its implications
- Master **measurement** techniques for different data types (areal vs point)
- Apply three major strategies for **handling spatial dependence**
- Implement solutions using Python spatial analysis ecosystem

Module Structure

1. **Foundations** — Theory, types, and consequences
2. **Measurement Methods** — Areal data indices and point data variograms
3. **Spatial Strategies** — Feature engineering, model structure, regularization

Part 1: Foundations

Spatial Dependence

Definition:
Spatial dependence (or **spatial autocorrelation**) exists when the value of a variable at one location is correlated with values at nearby locations, violating the independence assumption of classical statistics.

$$\text{Cov}(Y(s_i), Y(s_j)) \neq 0 \text{ for } s_i \neq s_j$$

Intuitive Interpretation:

> *Everything is related to everything else, but near things are more related than distant things* — **Tobler's First Law of Geography**

What produces spatial dependence?

A clustered map has **many possible causes** — and they call for different models and different policies. It helps to sort them into **two families**:

- **Interaction through space** — a value at one place *actually influences* its neighbours (contagion, spillovers, flows).
- **A shared spatial driver** — places merely *look alike* because something underneath varies smoothly (gradients, hidden confounders).

Telling these apart is the whole game: same hotspot, different story, different fix.
The next two slides unpack each — then we put them to work on a crime example.

Mechanisms (1) — interaction through space

Here the outcome genuinely **travels**: what happens *here* changes what happens next door. Same intuition, four flavours:

- **Diffusion** / **contagion**: spreads by contact or over time.
  Flu moving block to block; a crime wave. The outcome feeds back on itself — **WY**.
- **Spillovers** (contextual): a neighbour's *attributes* shape your outcome, with no feedback loop. *A good school nearby lifts your house price.* — **WX**.
- **Social** / **behavioural networks**: influence rides social ties, not raw distance.
  Solar panels adopted along friendships, not fences.
- **Transport** / **flow**: movement along a network or medium, often directional.
  A pollution plume downwind; contamination carried down a river.

Mechanisms (2) — a shared spatial driver

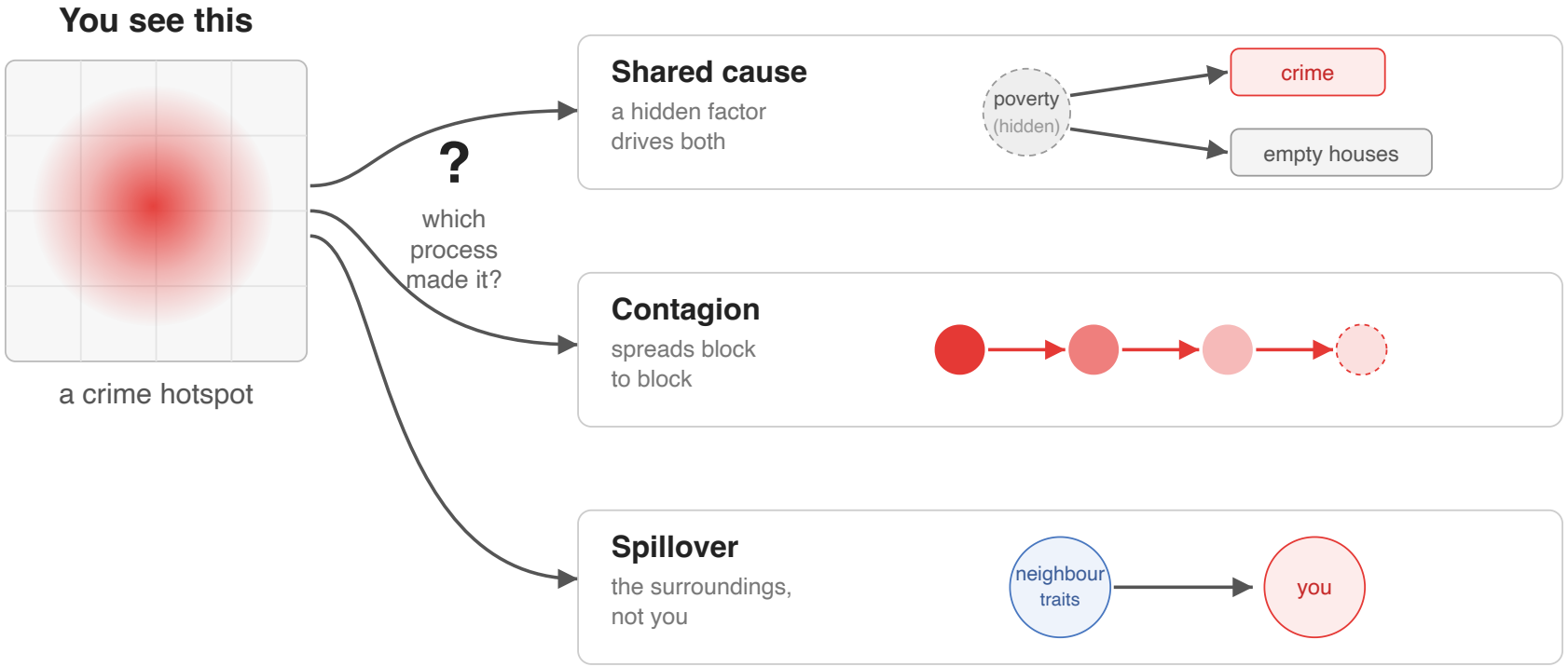
Here the neighbours never talk to each other. They only *look* alike because the same thing sits underneath all of them — like houses on one street sharing the same weather. Nobody is influencing anybody; a background factor makes them similar.

- **A smooth backdrop** (**environmental gradient**): climate, elevation or soil change gradually across the map, so nearby places get similar values.
  More rain → better crops; yields rise and fall with the rainfall.
- **A hidden common cause** (**shared-context confounding**): one unmeasured factor pushes *two* things up at once, so they look linked even though neither causes the other. *In poor areas both crime and empty houses are high — poverty drives both; fixing up the empty houses alone would not cut the crime.*

The tell: this "dependence" is really a **missing ingredient**. Measure that backdrop or hidden cause, add it to the model, and the spatial pattern often disappears — the very first check on the next slide.

Same hotspot — three suspects

You map crime and find a **cluster**. Looking at the map alone you **cannot tell why** — the three mechanisms we just met all produce the *same* picture.

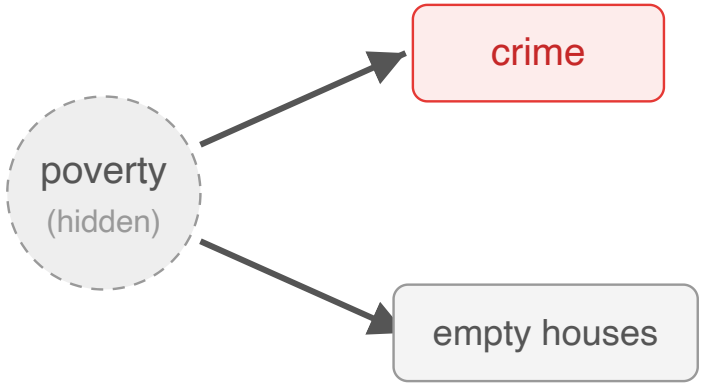


So don't guess from the map — **run a diagnostic**. Each suspect leaves a different fingerprint in the data (next slide).

Suspect 1 — Shared cause *(confounding)*

A hidden factor drives crime **and** other problems at once. No place influences
another — they just sit in the same bad context.

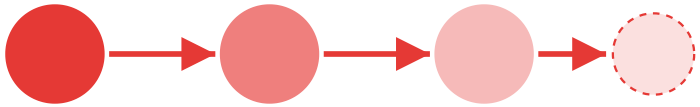
- **The tell** — the cluster **vanishes** once you add poverty / unemployment to the model.
- **The fix** — target the **root cause**: jobs, services, investment.
- **Careful** — treating the symptom (boarding up empty houses) won't move crime.



Suspect 2 — Contagion *(diffusion)*

Crime **spreads**: one incident raises the odds of the next one nearby, like an
infection jumping between blocks.

- **The tell** — the cluster **grows over time**, or travels along social ties.
- **The fix** — **interrupt transmission**: hotspot policing, focused deterrence, outreach.
- **Signature** — a wave that moves; yesterday's hotspot predicts today's.

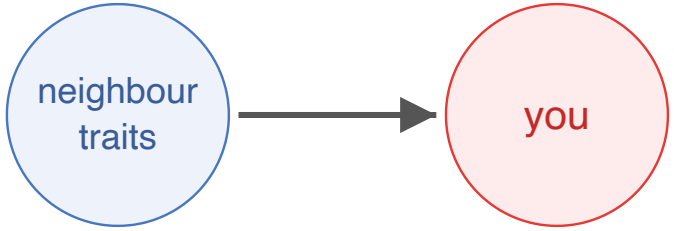


spreads block to block

Suspect 3 — Spillover *(contextual)*

Your risk is set by **what surrounds you** — not by your neighbours' crime itself.
The setting leaks in.

- **The tell** — neighbours' *traits* (**WX**) predict crime, but their *crime* (**WY**) does not.
- **The fix** — improve the **local setting**: lighting, upkeep, amenities.
- **Notation** — **WX** = neighbours' **characteristics**, **WY** = neighbours' **outcome**.



The point

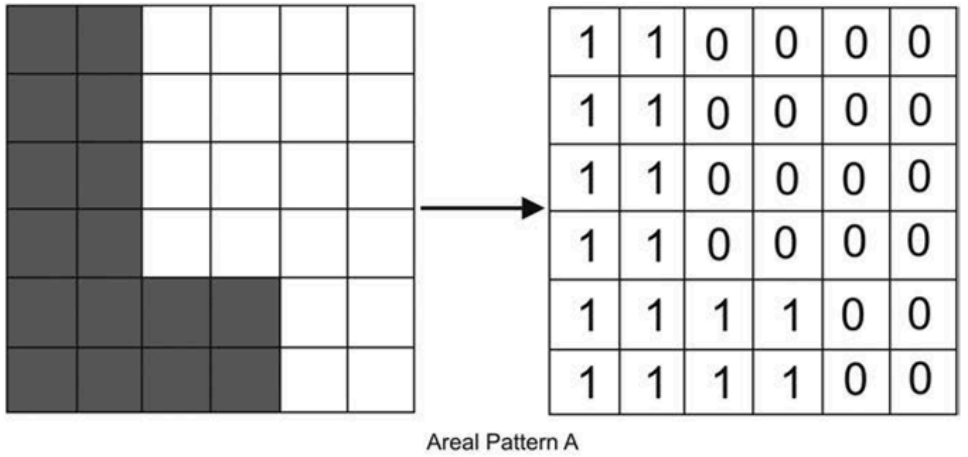
Same map, three stories. You cannot pick the right lever by eye — a **diagnostic** does. The tools coming up in this course *are* those diagnostics:

- **W** (spatial weights) — first defines who counts as a "neighbour". Every test needs it.
- **Moran's I on the residuals** — after adding poverty, does clustering drop to ~0?
  → **Suspect 1, shared cause**.
- **Spatial lag model** — is the neighbours' *crime* (**WY**) significant?
  → **Suspect 2, contagion**.
- **WX term** (SLX model) — are the neighbours' *traits* (**WX**) significant instead?
  → **Suspect 3, spillover**.

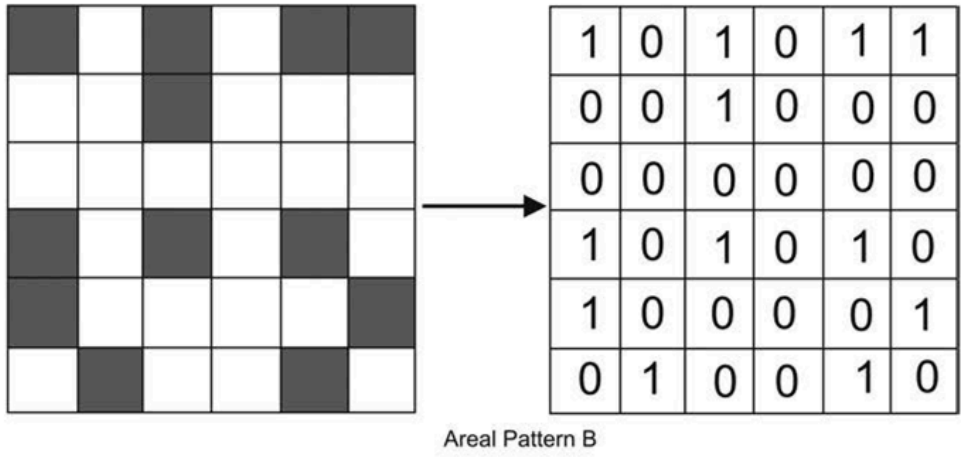
So each upcoming tool runs one of the three tests. The suspects were the **intuition**; these are the **instruments**.

Types of Spatial Dependence

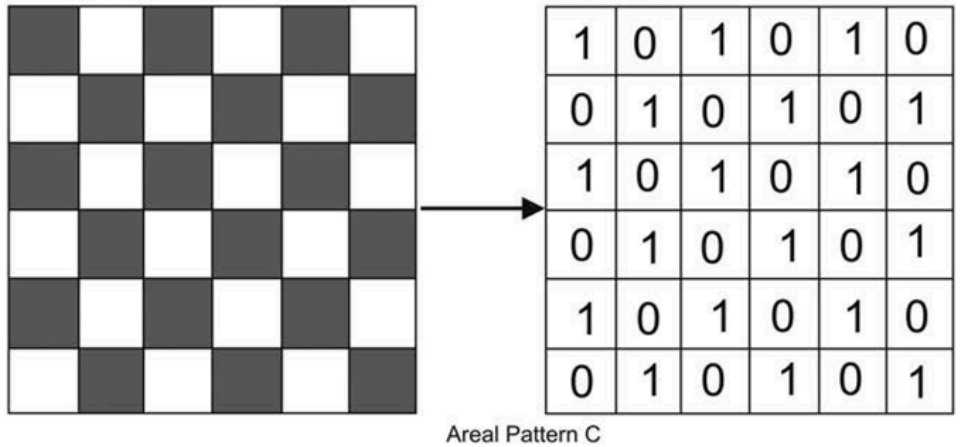
- **Positive:** Similar values cluster together
  - **Examples**: Housing prices, disease rates, pollution levels
- **Negative:** Dissimilar values are neighbors (rare)
  - **Examples:** Competitive retail locations, territorial animals
- **No Dependence:** Random spatial distribution
  - **Null hypothesis** in most spatial analyses



Areal Pattern A



Areal Pattern B



Areal Pattern C

Consequences of ignoring it — first, the model

Classical regression (**OLS**) rests on a quiet assumption: the errors are **independent** — one place's surprise says nothing about its neighbour's.

$$\mathrm{Cov}(\varepsilon_i, \varepsilon_j) = 0 \quad \text{for } i \neq j$$

Tobler's law breaks exactly this. How much trouble it causes depends on **where** the dependence sits (Anselin, 1988):

- in the **errors** only → a *nuisance* — the **context** / **confounding** suspect;
- in the **outcome** itself → *substantive* — the **contagion** suspect.

Same symptom, two very different diagnoses.

Case 1 — dependence in the errors *(a nuisance)*

The model's structure is right, but nearby errors move together — an unmeasured spatial factor you left out.

- **Coefficients stay unbiased** — the estimated effects are still centred on the truth.
- **Standard errors are wrong** — usually **too small** → inflated **t**-statistics → you flag effects as "significant" that are not. *False confidence.*
- **Why**: correlated neighbours repeat information — your **effective sample size** is smaller than **n**. Fifty listings on one block are not fifty independent facts.

Fix: correct the *inference* — a spatial-error model, or spatially-robust standard errors (Anselin, 1988).

Case 2 — dependence in the outcome *(substantive)*

Now the outcome truly depends on the **neighbours' outcome** (**WY**) — the contagion suspect. Leaving that term out is **omitting a real variable**.

- **Coefficients are biased *and* inconsistent** — the estimated effects themselves are wrong, not merely their uncertainty.
- **More data will not rescue you** — the bias does not shrink as **n** grows.
- This is the worse case: you would reach the **wrong scientific conclusion**.

Fix: change the *model*, not just the errors — a spatial-lag model (Anselin, 1988).

Then, the science — traps beyond the model

Even with a good model, spatial data invites reasoning errors:

- **Spurious correlation** — two variables share a common spatial trend and look linked though neither drives the other (spatial confounding — Suspect 1 again).
- **Ecological fallacy** — a relationship true for *areas* need not hold for *individuals* (Robinson, 1950). "Rich neighbourhoods" ≠ "rich people".
- **MAUP** (Modifiable Areal Unit Problem) — redraw the zones or change the scale and the result can shift (Openshaw, 1984). The boundaries are a **choice**, not a given.
- **Missed mechanism** — treat dependence as noise and you learn nothing about the process that produced it *(next slide)*.

Missed mechanism — the subtlest trap

The other three traps hand you a **wrong** answer. This one hands you **no** answer — your statistics can be flawless and you still learn nothing.
Clustering on a map was **produced** by a process. You can meet it two ways:

- **Correct it away** — patch the standard errors, report the coefficients, move on. The clustering is neutralised, never explained.
- **Explain it** — ask *which* process made it. That question **is** the finding.

Same crime map, two endings:

- **contagion** — a hot block drives its neighbours up → intervene locally, expect spillover.
- **confounding** — both blocks are simply poor → target poverty; spillover won't come.

> **Lesson —** spatial dependence is evidence about a process; "correcting it away" throws that evidence in the bin.

Challenge — and opportunity

Spatial dependence is **both**:

- a **challenge** (methodological) — it breaks OLS's assumptions, so inference and even estimates can mislead;
- an **opportunity** (scientific) — modelled well, it **reveals the process** (diffusion, spillovers) and **sharpens prediction**: kriging and Gaussian processes turn the "problem" into information.

> The rest of the course is about turning the **challenge** into the **opportunity**.

Part 2: Measurements (Areal)

Spatial Autocorrelation

Definition: **Correlation between values at different areal units, weighted by spatial proximity**

Key Requirements:

- Spatial weights matrix W or Graph defining **neighbourhood relationships**
- Choice of W affects results $\rightarrow$ need for a sensitivity analysis

Two Main Approaches:

1. **Global measures** : Single statistic summarizing dependence across entire study area
2. **Local measures** : Identify specific locations contributing to global patterns

Spatial connectivity: Weight Matrix W vs Graph

Two equivalent perspectives for encoding spatial relationships:

- **Matrix view** (classic): Spatial weights matrix $W \in \mathbb{R}^{n \times n}$ with entries w_{ij} quantifying the strength of connection from j to i.
- **Graph view** (modern): A sparse network of nodes (observations) and edges (relationships) with edge weights. In libpysal: **libpysal.graph.Graph**.

Why prefer Graph in practice?

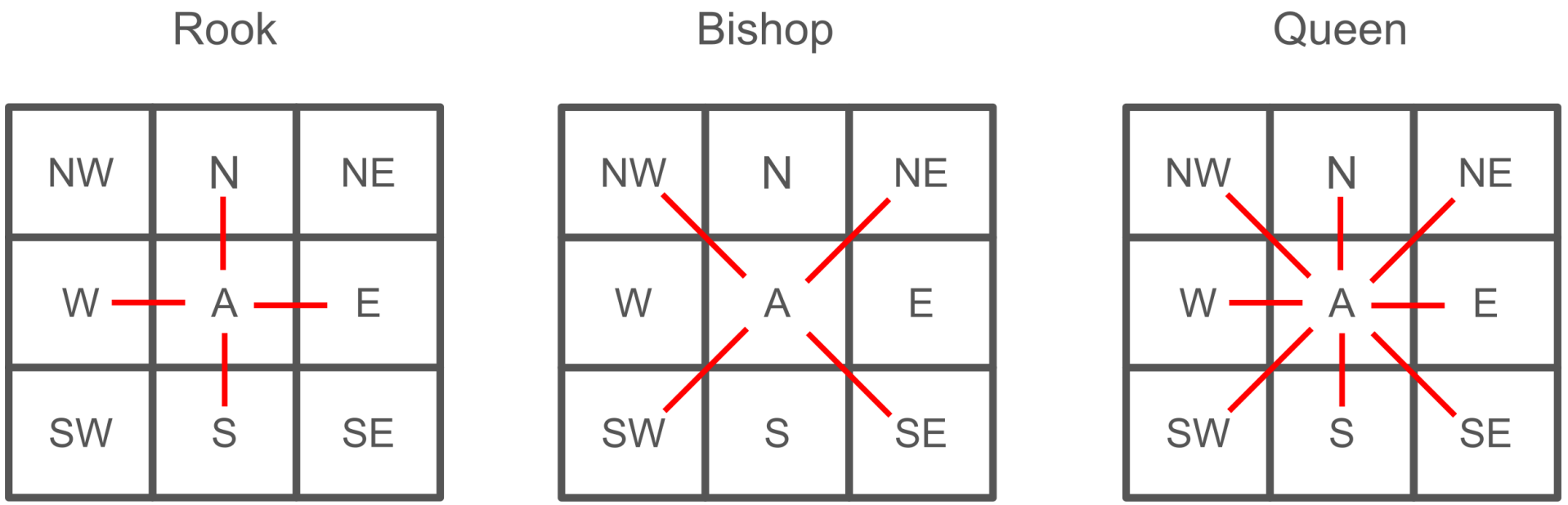
- **Natural API** for construction, inspection, plotting, and set operations
- **Efficient sparse representation**; easy access to neighbours, degrees, components, asymmetry
- **Simple normalization** via **transform** and seamless plotting, NetworkX export
- **Backward compatible**: can produce a legacy W object for packages that require it

> Key equivalence: the matrix W is just the adjacency matrix of the spatial graph, possibly after a transformation (e.g., row-standardization).

Building spatial graphs (libpysal >= 4.9)

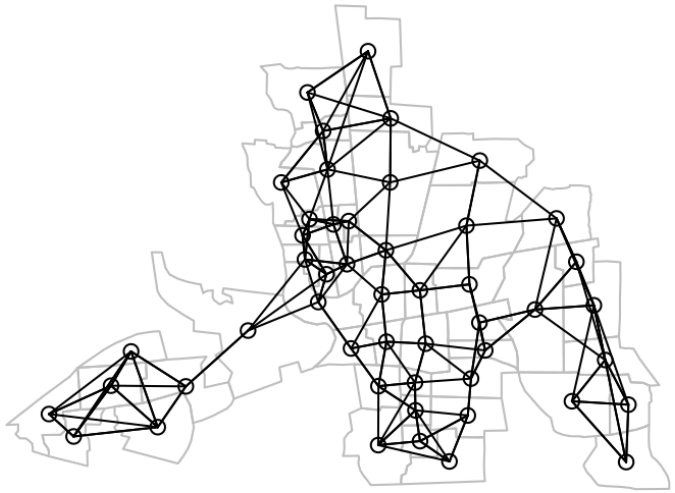
Common constructions (all produce undirected graphs unless noted):

- **Contiguity** (areal polygons)
  - **Rook**: neighbours share a border
  - **Queen**: neighbours share a border or a vertex



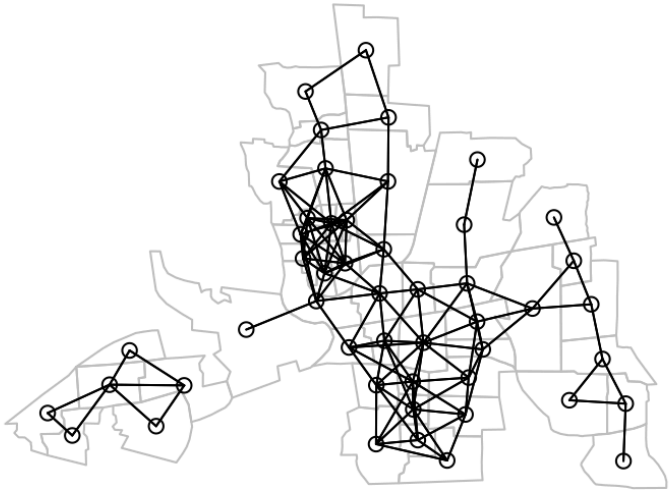
- **K-nearest neighbours** (KNN; points/centroids)
  - Each node connects to its k closest neighbours (can be asymmetric)

K nearest neighbours, k = 4



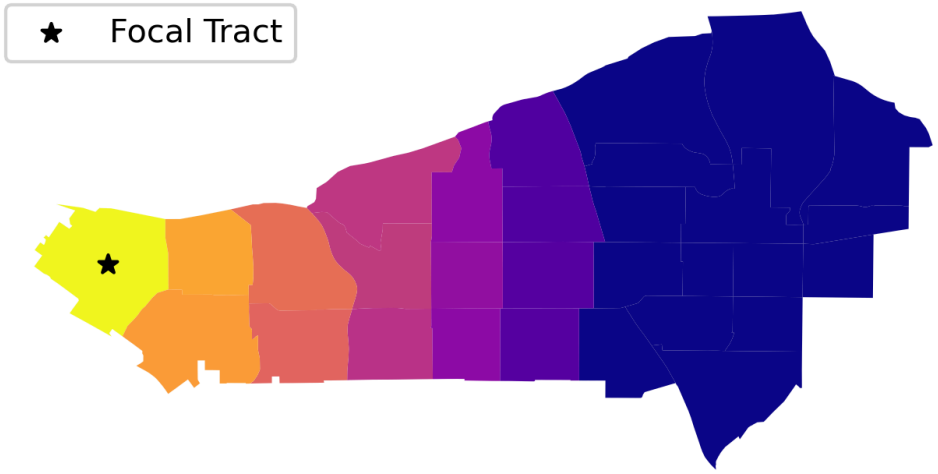
- **Distance band** (points or polygon centroids)
  - Edge exists if distance $\leq$ threshold (CRS units)

Distance based neighbours 0-0.6188642

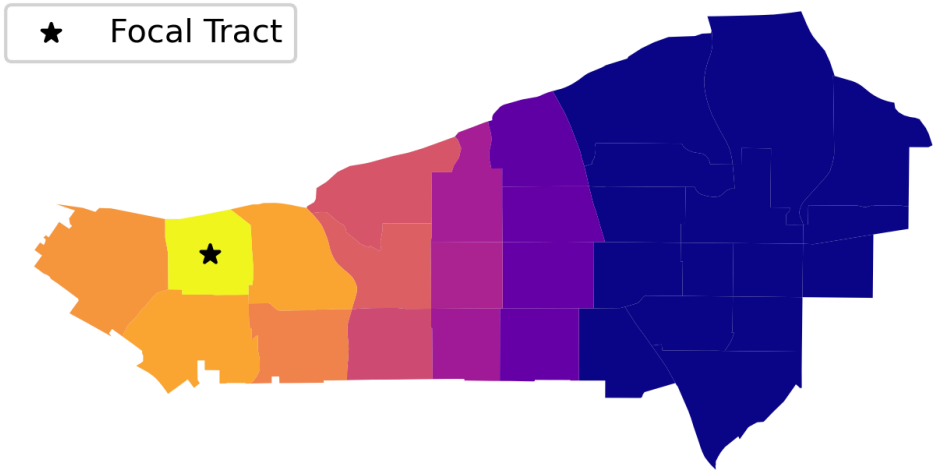


- **Kernels** (distance-decay)
  - Continuous edge weights decreasing with distance (e.g., triangular, gaussian)

Kernel centered on first tract



Kernel centered on 18th tract



```python
from libpysal.graph import Graph

# gdf: GeoDataFrame of polygons; use gdf.centroid for point-based rules

g_rook   = Graph.build_contiguity(gdf)                # rook by default
g_queen  = Graph.build_contiguity(gdf, rook=False)    # queen
g_band = Graph.build_distance_band(gdf.centroid, threshold=1000)  # m, projected
g_knn10  = Graph.build_knn(gdf.centroid, k=10)
g_kernel = Graph.build_kernel(gdf.centroid, kernel='triangular', bandwidth=1000)

# Row-standardize
g_rs = g_rook.transform('r')  # sum of outgoing weights for each node equals 1
```

Notes:

- Use a **projected CRS** for distance-based graphs (meters/feet). For polygons, graphs use centroids for distance rules.

> **Notebook: `notebooks/01_spatial_weights.ipynb`** — classic vs modern (**`Graph`**) API, contiguity/KNN/kernel/distance-band on San Diego tracts.

Inspecting and validating graphs

Before trusting any spatial statistic, sanity-check the graph you built — its neighbours, degrees, connectivity, and symmetry.

- **Neighbours** and **weights**
  - **g[unit_id]** or **g.adjacency.loc[unit_id]** $\longrightarrow$ neighbour weights (pandas Series)
  - **g.neighbours[unit_id]**, **g.weights[unit_id]** $\longrightarrow$ tuple views
- **Degrees** and **sparsity**
  - **g.cardinalities** $\longrightarrow$ degree per node; **g.pct_nonzero** $\longrightarrow$ sparsity of W
- **Connectivity** and **isolates**
  - **g.n_components**, **g.isolates**, **g.component_labels**
- **Asymmetry** (KNN)
  - **g_knn10.asymmetry()** lists edges lacking reciprocals
- Matrix forms and **export**
  - **g.sparse** $\longrightarrow$ SciPy CSR; **g.to_networkx()** $\longrightarrow$ NetworkX graph

Transformations (aka standardizations)

In spatial econometrics it's common to transform weights:

- **Row-standardization** ("r"): $w_{ij}^* = w_{ij} / \sum_j w_{ij}$
- **Double-standardization** ("d"): weights across all pairs sum to 1
- **Binary** vs **continuous**: contiguity is binary; kernels or border-length allow continuous weights

```
g_r = g.transform('r')       # row-standardized
g_d = g.transform('d')       # doubly-standardized
```

Remember: **Graph** is immutable; reassign to keep transformed versions.

Advanced graph concepts (brief)

Beyond contiguity and distance, a graph can encode composition, movement, and flows:

- Set operations (**composability**)
  - E.g., "Bishop" = Queen minus Rook: **bishop = g_queen.difference(g_rook)**
- **Network-based distances** (walk/drive times)
  - Build graphs from travel cost matrices (e.g., from OSM networks)
- **Flow-based graphs**
  - Build from observed flows (trade, commuting) as weighted, possibly directed edges
- **Coincident nodes** (points at exact same location)
  - Strategies: **jitter** (random displacement) or **clique** (connect co-located nodes)

These provide realistic alternatives to Euclidean proximity when infrastructure or movement data drive interactions.

Choosing and justifying a spatial graph

- **Match mechanism**:
  - **edge-sharing (contiguity)**: use rook/queen when transmission happens across shared borders (policy spillovers, adjacency diffusion).
  - **distance / transport**: use distance bands, KNN, or travel-time kernels when influence decays with separation or travel cost (accessibility effects).
  - **flows**: use observed flow matrices (directed/weighted) when measured exchanges (commutes, trade, migration) drive interactions.
- **Sensitivity analysis**: test conclusions across multiple reasonable graphs
- **Standardize consistently**: row-standardize when interpreting SAR/SDM impacts; document your choice
- **Check connectivity/islands**: isolates may require KNN or higher thresholds

Reference and examples: **knaaptime** "The Spatial Graph" tutorial

Moran's I — Global Spatial Autocorrelation

Moran's I is the spatial analogue of a correlation coefficient: it cross-multiplies each area's deviation from the mean with its neighbours' deviations, normalized by the total connection weight $S_0 = \sum_i \sum_j w_{ij}$.

$$I = \frac{n}{S_0} \frac{\sum_i \sum_j w_{ij}(y_i - \bar{y})(y_j - \bar{y})}{\sum_i (y_i - \bar{y})^2}, \qquad S_0 = \sum_i \sum_j w_{ij}$$

Intuitive Interpretation:

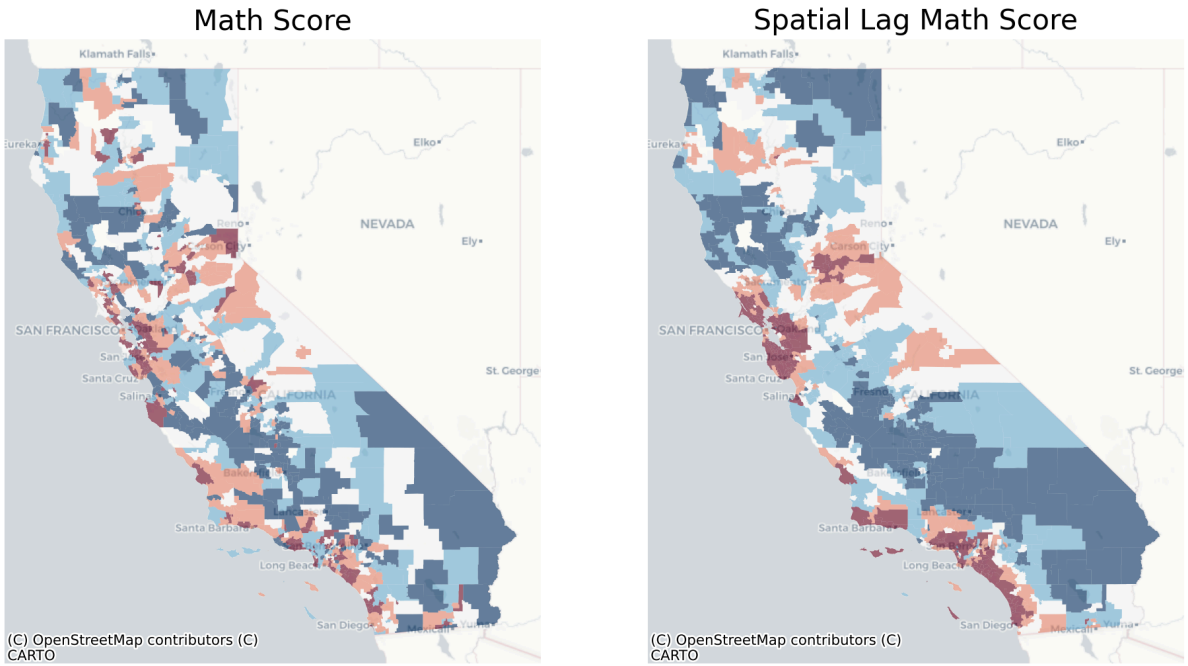
- Measures tendency for similar values to cluster spatially
- Compares observed covariation with the expected under spatial randomness

Moran's I — range and interpretation

- $I \in [-1, 1]$ approximately (exact bounds depend on W)
- $I > 0$: **Positive autocorrelation** (similar values cluster)
- $I < 0$: **Negative autocorrelation** (dissimilar values cluster)
- $I \approx 0$: **No spatial autocorrelation** (random pattern)

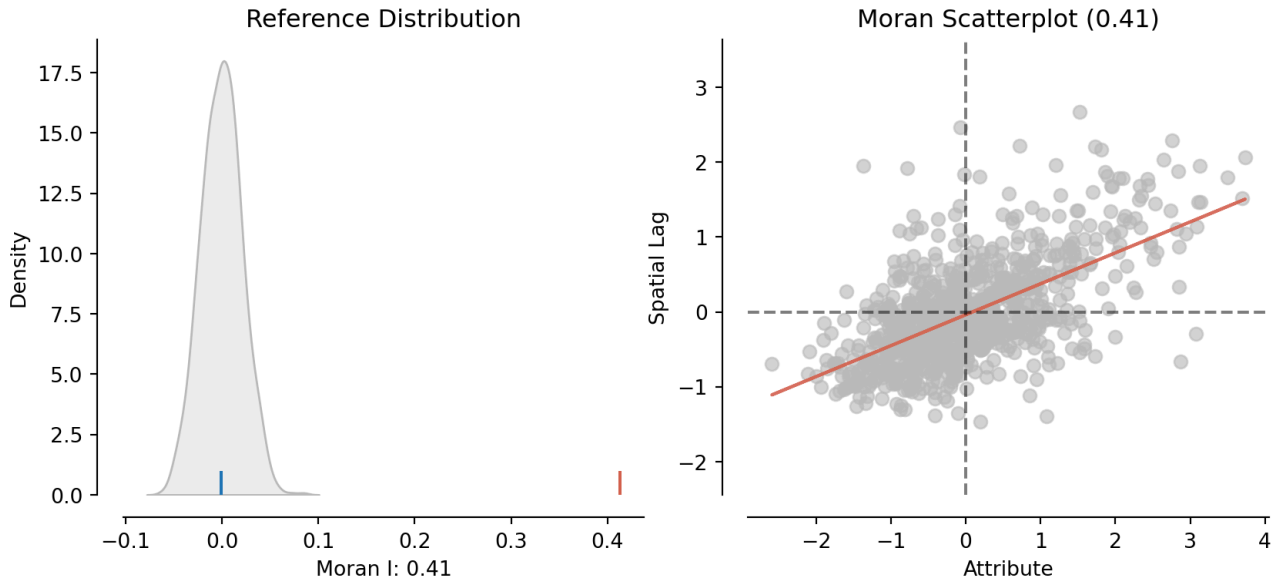
Global Moran's I — Visual diagnostics

- **Spatial lag map**: the neighbours' average (only when W is row-standardised); otherwise Wy is a weighted sum.
  - Visual cue for clustering vs randomness (a **spikier** surface)



> **Lesson —** map Wy, not just y: when the lag map echoes the raw map, neighbours mirror each other — the visual signature of positive autocorrelation.

- **Reference distribution**: the observed I is compared to a distribution built by randomly permuting the data across locations (**Complete Spatial Randomness, CSR**).
    - It shows what I is expected under no spatial autocorrelation; a **pseudo p-value** quantifies how unlikely the observed I is under that null.
- **Moran scatterplot** (Anselin, 1996): standardised y vs standardised Wy; slope $\approx I$



> **Lesson —** significance is judged against *shuffled* maps: if almost no reshuffling beats your I, the clustering is unlikely to be chance.

Worked example — Moran's I on San Diego home values

Global Moran's I on median house value across San Diego tracts (the data behind the figures above), Queen contiguity, row-standardised — the **knaaptime** ESDA workflow.

```
from libpysal.graph import Graph
from esda import Moran

y = tracts["median_house_value"]
w = Graph.build_contiguity(tracts, rook=False).transform("r").to_W()

mi = Moran(y, w, permutations=999)
print(round(mi.I, 3), mi.p_sim)      # 0.647   0.001
```

I = 0.647, **p_sim = 0.001**: strong, highly significant positive autocorrelation — expensive tracts sit next to expensive tracts.

> **Lesson —** a global statistic answers *whether* space matters, not *where*; a significant **I** is the cue to go local (LISA), never the end of the analysis.

> **Notebook: notebooks/02_spatial_autocorrelation.ipynb** — this San Diego example end-to-end (Moran, LISA, FDR).

Moran's I — Variations and Extensions

Moran's I has a local cousin: LISA decomposes the global statistic into one score per location, so we can map *where* clustering happens, not just whether it exists.

Local Moran's I (LISA):

$$I_i = z_i \sum_j w_{ij}\, z_j, \qquad z_i = \frac{y_i - \bar{y}}{s}$$

with s the sample standard deviation, so $\sum_i I_i = I$ up to a constant (Anselin, 1995).

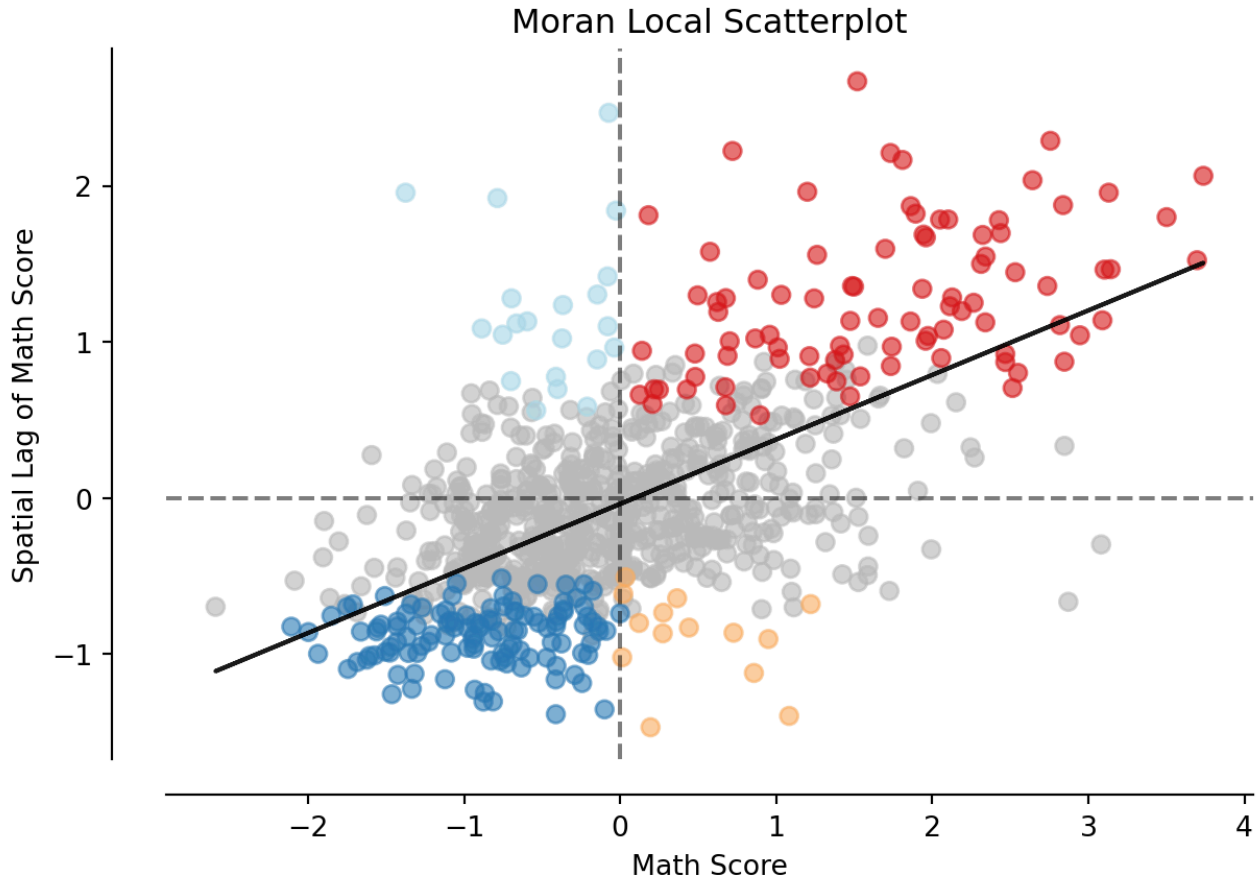
- Identifies **local clusters** and **spatial outliers**
- Four categories: **HH** (high-high), **LL** (low-low), **HL** (high-low), **LH** (low-high)

Interpretation

- HH/LL: local clusters (**hotspots** / **coldspots**)
- HL/LH: spatial outliers (**diamonds in the rough** or **holes in a donut**)

LISA — the local Moran scatterplot

- **Local Moran scatterplot**: highlights significant observations by quadrant
  - x axis: standardised value $z_i = (y_i - \bar{y})/s$;
  - y axis: spatial lag $\sum_j w_{ij} z_j$.
  - Points are **coloured by quadrant** (HH/LL/HL/LH) and **masked** when not significant (permutation p_{sim}).

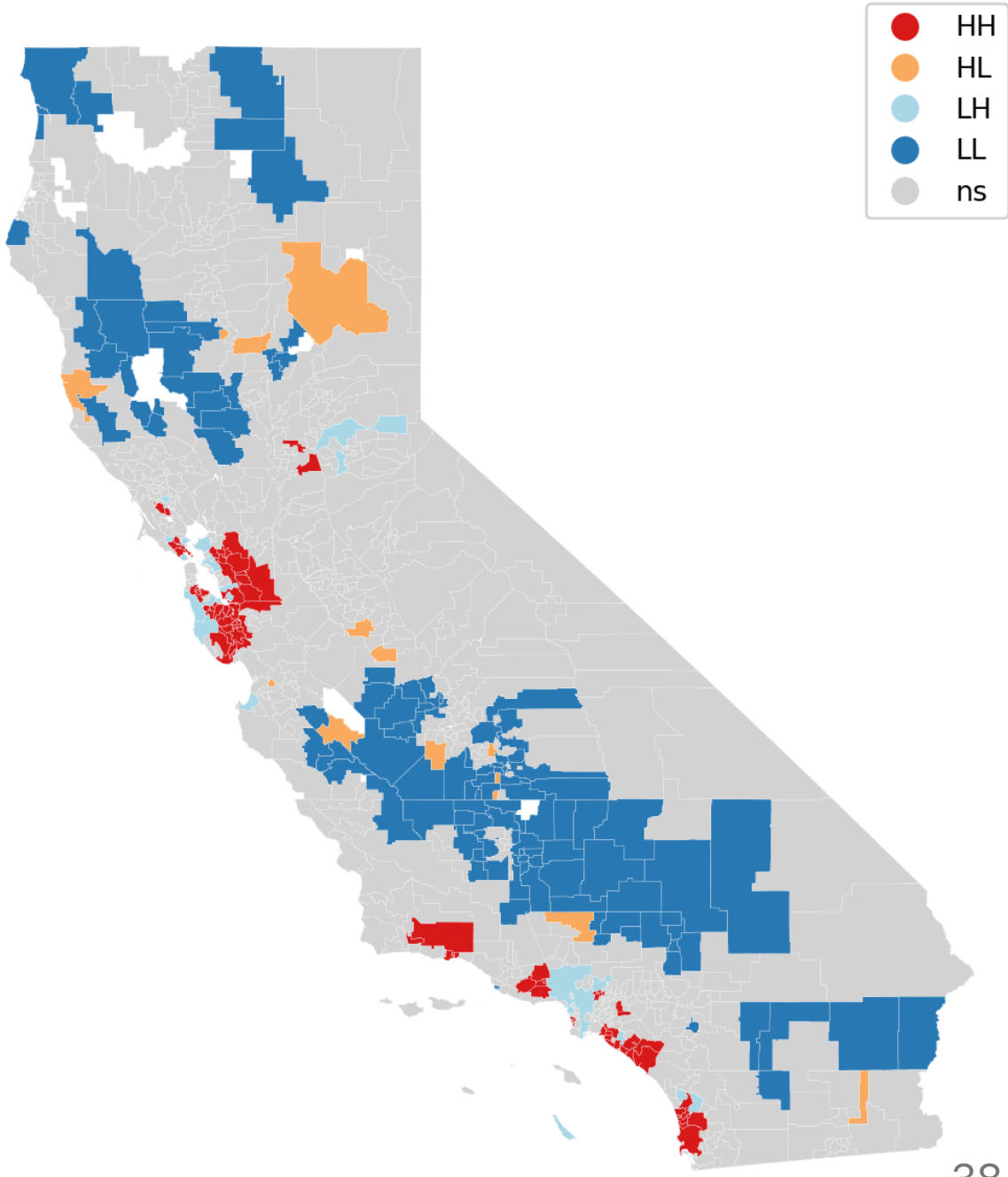


> **Lesson —** the quadrant names *what kind* of local association a place shows; the
> mask keeps only those a permutation test flags as unlikely by chance.

LISA — the cluster map

- **LISA cluster map** : shows HH, LL, HL, LH clusters; grey = not significant

> **Lesson —** the cluster map is the payoff — it names the *specific* places driving the
> global statistic, turning one number into an actionable geography.



Significance methodology (global and local)

The rigorous version of the pseudo-p you just met — skim if you only need the map, study if you will defend the inference.

Both the global and local Moran statistics are judged against a *simulated* null built by reshuffling the data. Three moving parts, unpacked next:

Overview

- Null: complete spatial randomness (CSR) / randomization — values are exchangeable over locations given W
- Permutations: build an empirical null and compute permutation p-values (p_sim) with +1 correction; choose sidedness
- Scope: Global Moran's I uses label permutations; LISA uses conditional permutations per location; apply multiplicity control for local tests (family-wise error rate, FWER / false discovery rate, FDR)

What's next (details)

- Mechanics of permutation p-values; global test design; local conditional permutations; multiple-testing control; reporting & best practices

Significance — the permutation null

If location were irrelevant, any reshuffling of the values across areas would be as likely as the one we observed. We shuffle many times, rebuild the statistic, and ask how extreme the real value looks (Hope, 1968).

- Null (randomisation / CSR): values are exchangeable over locations, given the graph W
- Mechanics: for R permutations, shuffle the labels and recompute the statistic $T^{(r)}$
- Pseudo p-value with the **+1 correction** (so it is never zero):

$$p = \frac{1 + \#\{r : |T^{(r)}| \geq |T_{obs}|\}}{R + 1} \quad \text{(two-sided)}$$

$$p_{\text{right}} = \frac{1 + \#\{r : T^{(r)} \geq T_{obs}\}}{R + 1} \quad \text{(one-sided)}$$

Permutation p-values — resolution & choices

How many shuffles, and which p-value? The permutation count sets both the finest p you can resolve and the noise in the estimate.

- Minimum attainable p is $1/(R+1)$ → use larger R (e.g. 9,999) for finer resolution
- Estimate noise: $\mathrm{SE}(\hat{p}) \approx \sqrt{p(1-p)/(R+1)}$ → more permutations stabilise small p
- Prefer permutation p-values (**p_sim**) over asymptotic ones (**p_norm**)
- Set and report a random seed for reproducibility

> **Lesson —** a pseudo p-value is only as fine as your permutation budget: report R, and never read a p below $1/(R+1)$ as "more significant".

Global Moran's I — test design details

For the global statistic the permutation is unconditional: shuffle the whole vector, recompute I. The only choices — permutation count and tail — follow your hypothesis, not the result.

- Statistic: $I = \frac{n}{S_0} \frac{\sum_i \sum_j w_{ij}(y_i - \bar{y})(y_j - \bar{y})}{\sum_i (y_i - \bar{y})^2}$, with $S_0 = \sum_i \sum_j w_{ij}$
- Permutation scheme: permute the vector y across locations (W fixed), recompute I each time
- Sidedness mapping:
  - Positive autocorrelation $\rightarrow$ right-tail test (large I)
  - Negative autocorrelation $\rightarrow$ left-tail test (small I)
  - Exploration/diagnostics $\rightarrow$ two-sided by default
- Practical choices:
  - R = 999 as a minimum; prefer 9,999 for publication-quality results
  - Row-standardize W (or Graph.transform('r')) for comparability across datasets
  - Report: I, p_sim, R, sidedness, and the W/Graph definition

Local Moran's I (LISA) — conditional permutations

For a *local* test we respect each area's own neighbourhood: hold location i fixed and shuffle the remaining values among its potential neighbours. This is what **esda.Moran_Local** does under the hood (Anselin, 1995).

- Local statistic: $I_i = z_i \sum_j w_{ij} z_j$ with standardised $z = (y - \bar{y})/s$
- Conditional randomisation (what **esda.Moran_Local** implements):
  - Hold location i (and its weights $w_i.$) fixed; permute the labels among the other locations; recompute I_i
  - Builds a location-specific null respecting the local neighbourhood and the value distribution
- Per-location p-value: two-sided on $|I_i|$, or one-sided by quadrant for a specific alternative (HH: right tail; LL: left tail)

Why false hotspots are unavoidable

A test at $\alpha = 0.05$ is a **bar set so that pure noise clears it 5% of the time** — that tolerance is the *definition* of the test, not a flaw. Run it once and you are probably fine. A LISA map runs **one test per area**, so that harmless 5% compounds.
Think of it as a lottery:

- **One ticket** at 5% odds → you almost certainly lose (correctly "not significant").
- **Buy 628 tickets** → you are almost certain to win *at least one* by pure luck.
- Each "win" is a **false hotspot** — and the map cannot tell luck from real signal.

If all **628** San Diego tracts were pure noise, you would *still* expect
$628 \times 0.05 \approx 31$ of them to light up "significant".

> **Lesson —** give noise enough independent chances and it *will* clear the bar somewhere; that is why a LISA map needs multiple-testing correction — not why the test is wrong.

Multiple testing for LISA — FWER (family-wise error rate)

A LISA map runs one test per area, so with n areas some "significant" hotspots appear by chance alone. If the tests were independent, $\mathrm{FWER} = 1 - (1 - \alpha)^n$; at $n = 100,\ \alpha = 0.05$ that is ≈ **0.994** (spatial dependence softens this, but the warning stands). FWER methods make even a single false hotspot unlikely — strict, for when each claimed cluster triggers a costly decision.

- Goal: control the probability of *any* false positive across all locations
- Bonferroni: test each at α/n (simple, conservative)
- Holm–Bonferroni (preferred): sort p, step-down, stop at first non-reject
- Use for: confirmatory settings, small n, high-stakes inference

Multiple testing for LISA — FDR (false discovery rate)

FDR is the gentler, more common choice for exploratory mapping: rather than forbidding any error it caps the *share* of flagged areas that are false, so you discover more at a controlled rate (de Castro & Singer, 2006 — the method **esda.fdr** implements).

- Goal: control the expected proportion of false discoveries
- Benjamini–Hochberg: sort $p_{(1)} \leq \cdots \leq p_{(n)}$; largest k with $p_{(k)} \leq (k/n)\, q$ → 1..k significant
- BH stays valid under the positive dependence typical of LISA; report q (e.g. 0.05)

On our San Diego example: **218** tracts pass raw $\alpha = 0.05$, but only **84** survive FDR (**esda.fdr**) — HH = 34, LL = 47.

> **Lesson —** never map raw LISA p-values: correction here removed ~60% of "significant" hotspots. Always report how many survived, and which rule you used.

Mapping LISA — a practical workflow

1. Compute **local p-values** by conditional permutations (R ≥ 999)
2. Apply **multiplicity correction** (Holm for FWER or BH for FDR)
3. **Classify HH/LL/HL/LH** using signs, then **mask by adjusted significance**
4. **Report** R, sidedness (encodes your alternative hypothesis, i.e., positive autocorrelation, negative autocorrelation, or any deviation), and correction; add a legend describing classes

> **Lesson —** a defensible LISA map is a *pipeline*, not a function call: permute, correct, classify, mask, report — skip a step and the hotspots are not trustworthy.

> Reference and examples: knaaptime “Spatial Autocorrelation” tutorial

Part 2: Measurements (Point Data)

Introduction

Context: Continuous spatial fields observed at **point locations** (no built-in neighborhood).
Goal: Summarize how **similarity decays with distance** so we can **predict** and **plan sampling**.
Use-cases

- Environmental monitoring (temperature, rainfall, air quality)
- Natural resources (ore grades, soil properties)
- Ecology (biomass, chlorophyll, moisture)

Key idea: Use **distance-based** measures of spatial continuity rather than adjacency lists.

Assumptions

Stationarity (local enough)

- Mean and correlation structure don’t change dramatically across the study area.
- If strong trends exist $\longrightarrow$ **detrend** first (e.g., regress on elevation, lat/lon smooths) and variogram the **residuals**.

Isotropy vs. Anisotropy

- **Isotropy:** dependence depends on **distance only**.
- **Anisotropy:** dependence also depends on **direction** (wind, valleys, rivers).
  $\longrightarrow$ Use **directional** semivariograms to detect/model it.

Practical note: The semivariogram is robust to mild mean drift (it uses **differences**), but strong trends should be handled before modeling.

Detrending — why remove the trend first

A **trend** is a systematic drift in the mean across space — *position itself* predicts the value. Kriging assumes the mean is roughly flat (stationary), so a strong trend breaks it.
Example — temperature vs. elevation:

- Temperature falls steadily as you climb. Two far-apart stations differ **mostly because one sits higher**, not from genuine spatial correlation.
- The variogram then tracks the *slope of the mountain*, not the dependence you want — it keeps climbing instead of leveling off.

Fix — split `value = trend + residual`:

1. Regress temperature on elevation → the deterministic drift.
2. Residual = observed − predicted → "warmer/cooler than altitude explains".
3. Variogram (and krige) the **residuals**; add the trend back to predict.

> **Lesson —** detrend when the mean drifts across space, then model the *residuals* so the variogram captures real spatial dependence — not the gradient. This is regression / universal kriging.

(Semi)Variogram — Motivation

Question: Up to what distance do nearby points still share useful information?
Semivariogram = distance $\rightarrow$ average dissimilarity

- Near 0: low (or a **nugget jump** if micro-noise exists)
- Increases as similarity decays
- Levels off at the **sill** (beyond this, points are effectively independent)

For a distance class (lag) **h**,

$$\gamma(h) = \frac{1}{2|N(h)|} \sum_{(i,j)\in N(h)} \big(z(s_i) - z(s_j)\big)^2$$

In simple words: Half the **average squared difference** of all pairs whose separation falls in that distance bin.

1. We create an **empirical curve** selecting 10-20 distance bins
2. We **fit a smooth model** (spherical/exponential/Gaussian/Matérn) to this empirical curve.

One dot = one bucket of *pairs*

The unit above is the **pair**, not the point: no dot on the curve comes from a single location.

Building the dot at $h \approx 250$ m (bin 200–300 m):

1. Scan **every pair** of samples; keep those separated by 200–300 m.
2. Square each kept pair's value difference, $(z_i - z_j)^2$.
3. Average over the bin, halve → **one dot**.

Meuse's **155** points give $\frac{155 \times 154}{2} =$ **11,935 pairs** in total, sorted into bins: each dot averages *hundreds* of comparisons, and **every point feeds many bins** — it pairs with near *and* far samples alike.

Why pooling is allowed: *stationarity* (position doesn't matter) and *isotropy* (direction doesn't matter) — the two assumptions that licence one bucket per distance.

> **Lesson —** a dot averages *all pairs* at that separation, so a bin needs enough pairs (≥ 30) to be trusted: no point owns a bin, and no dot rests on one comparison.

How to Compute It — Step by Step

The empirical semivariogram is a recipe, not a black box: bin every pair of points by their separation distance, average the squared differences inside each bin, and plot the result. These seven steps turn a scatter of measurements into the curve we then fit.

1. **Prepare the variable**
   Clean outliers; transform if heavily skewed (e.g., log).
2. **Work in metric CRS**
   Distances in meters/kilometers.
3. **Choose max lag**
   $\leq$ about ½ of the study extent (or max inter-point distance).
4. **Create 10–20 distance bins**
   Aim for **≥ 30 pairs/bin**; merge sparse tails.

How to Compute It — steps 5–7

Same recipe, continued: from the binned pairs to the plotted, annotated curve.

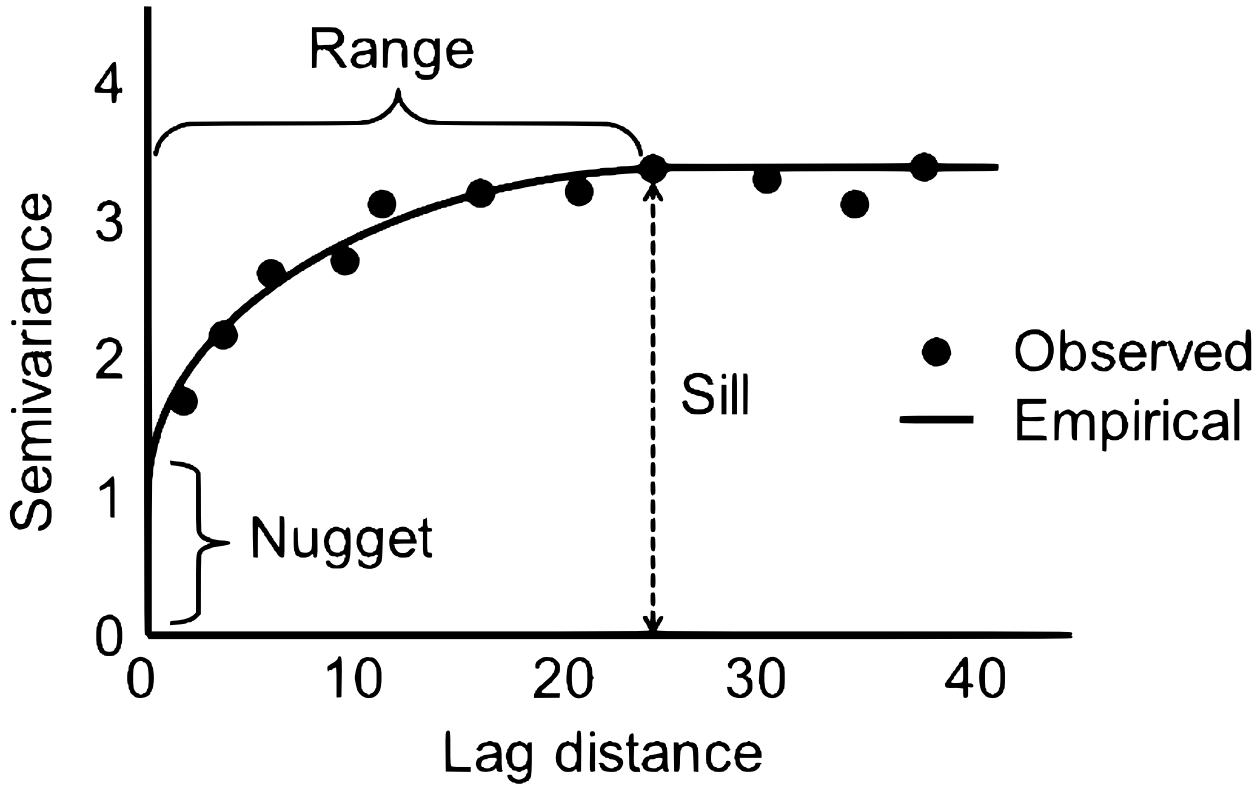
5. **(Optional) Directional bins**
   E.g., 0°, 45°, 90°, 135° to check anisotropy.
6. **Compute empirical $\gamma(h)$**
   Half the average squared difference of the pairs in each bin.
7. **Plot & annotate**
   Visually note small-h behaviour (nugget), rise, and levelling (sill).

Parameters

- **Nugget** : value at (almost) zero distance $\longrightarrow$ **micro-scale variability + measurement noise**.
- **Partial sill** : height of **spatial signal** above the nugget.
- **Sill (total)** : nugget + partial sill ≈ **total variance** of the process.
- **Range** : distance where $\gamma(h) \approx 95\%$ of sill $\longrightarrow$ beyond **correlation fades**.

Rules of thumb

- **Nugget/Sill** < 0.25 $\longrightarrow$ strong spatial dependence; > 0.75 $\longrightarrow$ weak.
- Sensor spacing ≈ **~** $\frac{1}{2}$ **range** is often efficient.



Sampling design — how far apart to place sensors

The **range** is how far spatial correlation reaches: points farther apart than the range are effectively **independent**. So the range tells you how densely to sample.
Rule of thumb: spacing ≈ ½ the range. It steers between two opposite mistakes:

- **Spacing** $\geq$ **range** → neighbours are independent → gaps you *cannot* interpolate across; kriging falls back to the global mean. *Under-sampled*.
- **Spacing** $\ll$ **range** → sensors nearly measure the same thing → each new one adds little → wasted cost. *Over-sampled*.

At ½ **range**, every gap sits inside its neighbours' correlation reach — enough overlap to interpolate, without paying for redundant readings.
Circularity: you need the range to set spacing, but samples to estimate the range → run a small **pilot survey** (or use prior studies), then design the main campaign at ~½ range.

> **Lesson —** space samples at about half the correlation range: near enough that neighbours still inform each other, far enough to avoid paying for redundancy.

What is kriging?

Kriging = spatial interpolation. You measured a value at scattered points; you want it **everywhere else** — a continuous map (zinc at 155 Meuse samples → the whole floodplain). Predict each location as a **weighted average of nearby measurements**:

$$\hat{z}(s_0) = \sum_i \lambda_i \, z(s_i)$$

- The **weights λ_i come from the variogram**, not a fixed "closer = bigger" rule — the data's *own* correlation structure decides how much each neighbour counts.
- That is what beats plain **inverse-distance weighting**: kriging **learns** the weights.

Two outputs:

- a **prediction** at every location (the map);
- a **kriging variance** — per-location uncertainty: tight near samples, honest in the gaps.

> **Lesson —** kriging interpolates with weights the variogram supplies, returning both a map and a matching map of how much to trust it.

From Variogram to Kriging (How It's Used)

The variogram is not the destination — it is the input to interpolation. Its three parameters become operational knobs for kriging, as you would see fitting it to the **data/meuse/meuse.csv** zinc samples (**x, y, zinc**).

> **Notebook:** **notebooks/02b_variograms.ipynb** — Meuse zinc: fit (nugget ≈ 0.08, partial sill ≈ 0.63, **total sill ≈ 0.72**, range ≈ 1048 m; nugget/sill ≈ 0.12) → ordinary kriging → CV.

Workflow: empirical $\gamma(h) \rightarrow$ fit model $\rightarrow$ **kriging** $\rightarrow$ predictions + **kriging variance**.

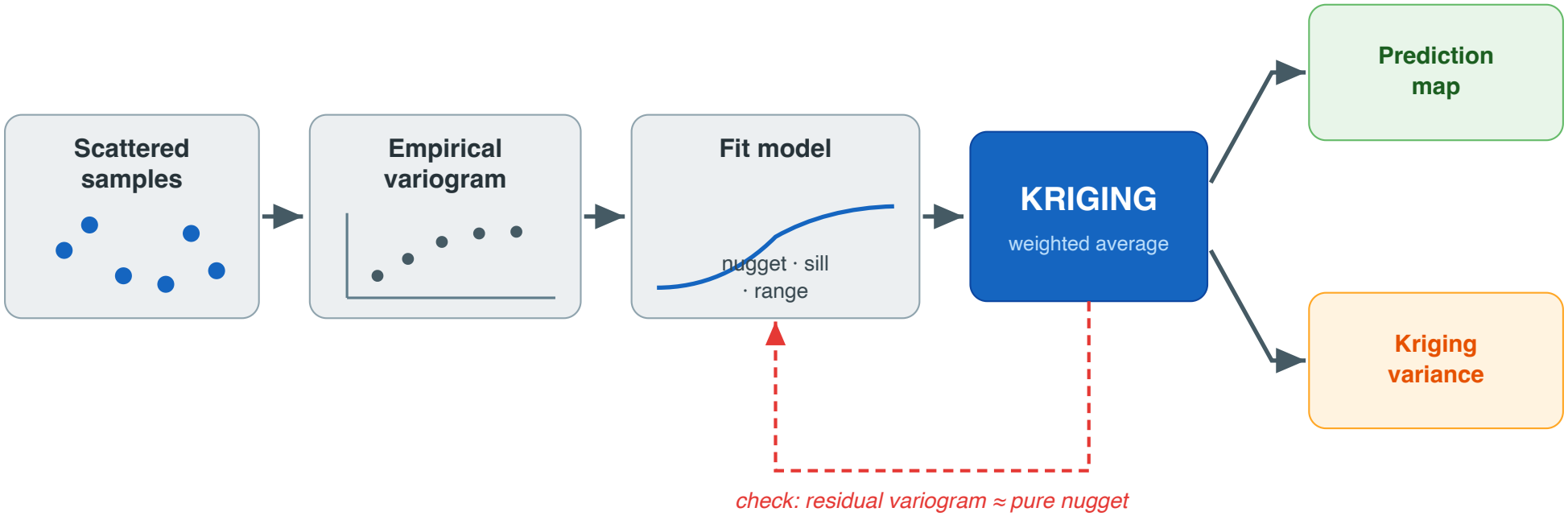
Parameter roles

- **Range**: size of the neighborhood that influences each prediction.
- **Nugget**: lower bound on prediction error (noise floor).
- **Partial sill**: strength of the spatial signal; influences how much neighbors matter.

Validation

- Use **spatially aware CV** (blocked or leave-location-out).
- After modeling, the **residual variogram** should be ≈ **pure nugget**.

The full pipeline — at a glance



> **Lesson —** one flow: samples → empirical variogram → fitted model (nugget, sill, range) → kriging → prediction **and** uncertainty; the residual variogram confirms the fit.

Best Practices & Takeaways

Best practices

- Metric CRS; sensible max lag; enough pairs per bin
- Check trend & anisotropy; if present, **detrend** or use **universal/regression kriging**
- Prefer **simple, plausible** models (spherical/exponential/gaussian) before Matérn
- Report **nugget, partial sill, sill, range**, and **nugget/sill ratio**
- Validate with **spatial CV**; report accuracy **and** interval coverage

Takeaways

- Semivariogram = **distance vs. average dissimilarity**.
- **Nugget** separates **noise/micro-scale** from structure.
- **Range** = practical **spatial horizon**; guides sampling.
- Keep models simple; focus on **interpretability + validation**.

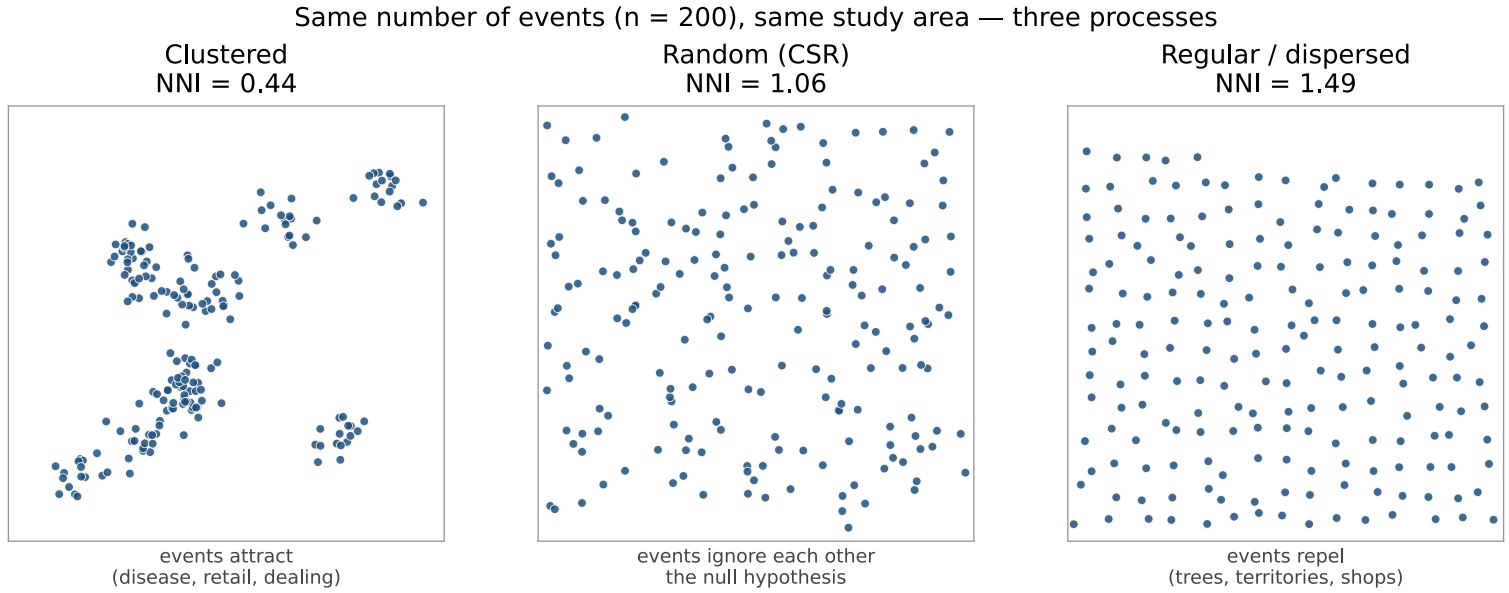
Point patterns (discrete events)

So far the **locations were given** and we studied a *value* attached to them — house value per tract, zinc per soil sample. Now the **locations themselves are the data**. There is no "value" to model: the events are trees, crimes, disease cases, earthquakes, and the question is **where they chose to happen**.

- A **point pattern** = the set of event locations in a study window (with attributes → *marked point pattern*).
- The **window matters**: "12 events" means nothing until you say *in how much space*.
- We ask three questions, in this order:
  i. Are events **clustered**, **regular**, or **random**? — and *compared to what?*
  ii. At what **scale** does the pattern appear?
  iii. Is it **interaction** between events, or just a **busy part of town**?

> Question 3 is the hard one, and the one this section builds toward.

The only question: clustered, random, or regular?



Same n, same window, three processes. The eye sees it — but "it looks clustered" is not a finding. **NNI** puts a number on it: mean nearest-neighbour distance ÷ the distance expected **under randomness**. **NNI < 1** clustered · **≈ 1** random · **> 1** regular. Every method here is that one move: **measure something, compare it to what randomness would have given**.

> **Lesson —** a point pattern is never "clustered" in the absolute; it is clustered *relative to a null model*, so the null is the part you must state out loud.

CSR — the null everything is measured against

Complete Spatial Randomness = the **homogeneous Poisson process**. Two properties — keep them apart:

1. **Uniform intensity** — expected events $= \lambda$ per unit area, the **same everywhere**. No place is special.
2. **Independence** — events do not influence one another.

So $N(A) \sim \text{Poisson}(\lambda|A|)$. A pattern can break CSR by failing **either** one — and they mean completely different things:

What broke	Story	Example
Independence	events *interact*	contagion; a dealer attracts dealing
Uniform intensity	the *place* varies	more crime where more people are

A test that rejects CSR does not tell you which one broke.

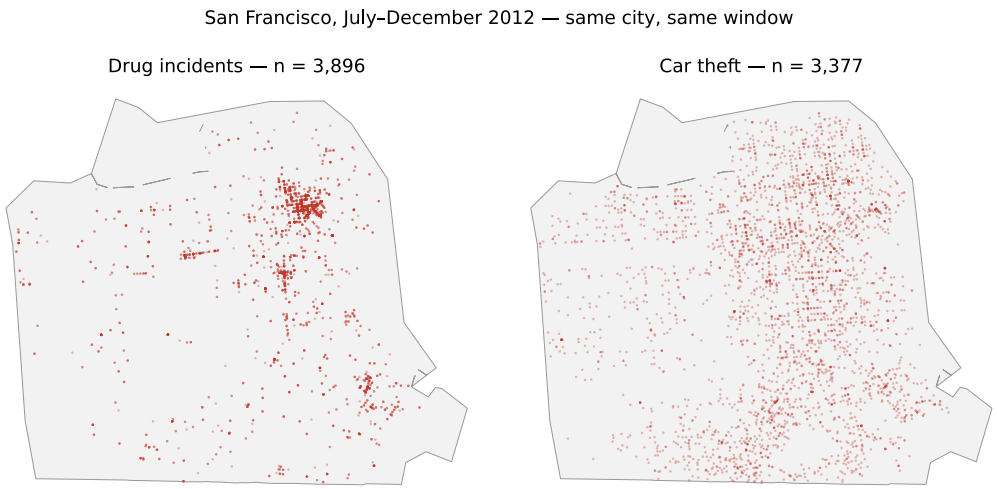
> **Lesson —** CSR bundles two assumptions — constant intensity *and* independence — so "we rejected CSR" begins the analysis, it does not end it.

Our data: SF crime, Jul–Dec 2012

Two crime types, one city, six months (**libpysal SanFran Crime**):

- **Drugs** — $n = 3{,}896$, $\lambda = 31.6$/km² · **Car theft** — $n = 3{,}377$, $\lambda = 27.4$/km²
- Window: SF land boundary, **123.4 km²**, 805k residents.

Similar counts and intensity — **very different patterns**. Drugs collapse onto a few blocks (the Tenderloin); car theft spreads across the city.
Both will "reject CSR", and they will mean **different things by it**.



First-order: where are events dense?

First-order = variation in **intensity** $\lambda(s)$ — the absolute location of events. It answers **where**, never **why**.

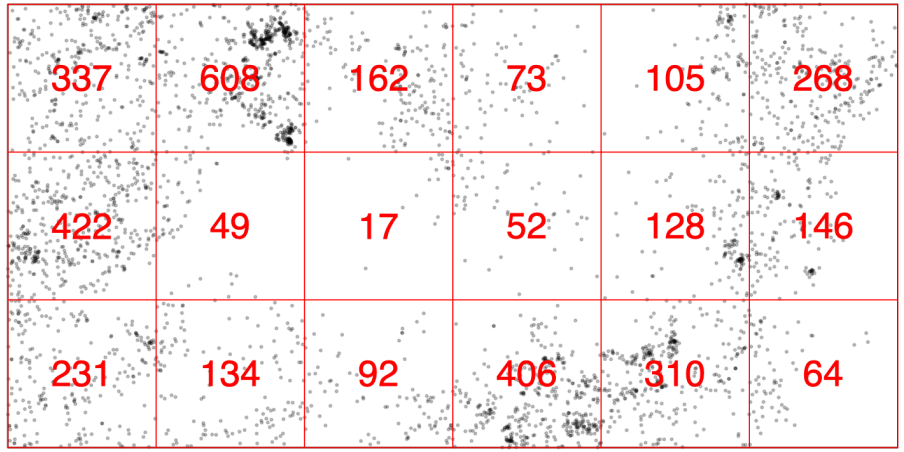
- **Quadrat methods** — partition the window, count, compare counts to Poisson.
- **Kernel density (KDE)** — a smooth intensity surface.

Second-order (later) = **interaction** between events — whether one event's presence changes the chance of another nearby.

> The two are genuinely different questions, and — as the last slide of this section shows — **a first-order effect can masquerade as a second-order one**.

Quadrat method

- **Procedure**:
  i. Divide the study region into equal-area quadrats.
  ii. Count the number of points per quadrat.
  iii. Compute local intensity $\hat{\lambda}(A_l) = \frac{n(A_l)}{|A_l|}$.
- **Insights**:
  - Larger quadrats → smoother maps.
  - Smaller quadrats → "spiky" results.
- **Limitations**:
  - Sensitive to size/shape (MAUP).
  - Captures **only first-order effects**.



Practical trap: only count cells that are **fully inside the window**. A cell that is
90% ocean records "few crimes" and is not evidence of anything.

Quadrat counts → is it random?

Counts alone don't answer "clustered or not." Compare them to **complete spatial randomness (CSR)**: if events fell independently at constant intensity, cell counts follow a **Poisson** distribution — whose defining property is **variance = mean**.
Variance-to-mean ratio (VMR):

$$\mathrm{VMR} = \frac{s^2}{\bar{n}}, \qquad \bar{n} = \frac{\sum_l n_l}{m}, \quad s^2 = \frac{\sum_l (n_l - \bar{n})^2}{m-1}$$

- **VMR ≈ 1 → random** (Poisson).
- **VMR > 1 → clustered** — some cells packed, many empty → variance inflated.
- **VMR < 1 → regular / dispersed** — counts nearly equal → variance suppressed.

Sanity check first. On 200 simulated CSR patterns in the SF window (53 fully-land cells): $\mathrm{VMR} = 1.00 \pm 0.20$. The statistic knows what random looks like.

> **Lesson —** a grid of counts becomes a test by comparing its spread to its mean: Poisson randomness has variance = mean, so VMR points you to clustered vs. regular.

Quadrat test — is the deviation significant?

Real numbers, SF drug incidents, 10×10 grid, 53 fully-land cells:

```
mean count = 68.6     variance = 46,305     VMR = 675
chi2 = 35,108   df = 52   p < 0.001
```

Index of dispersion / χ^2 goodness-of-fit:

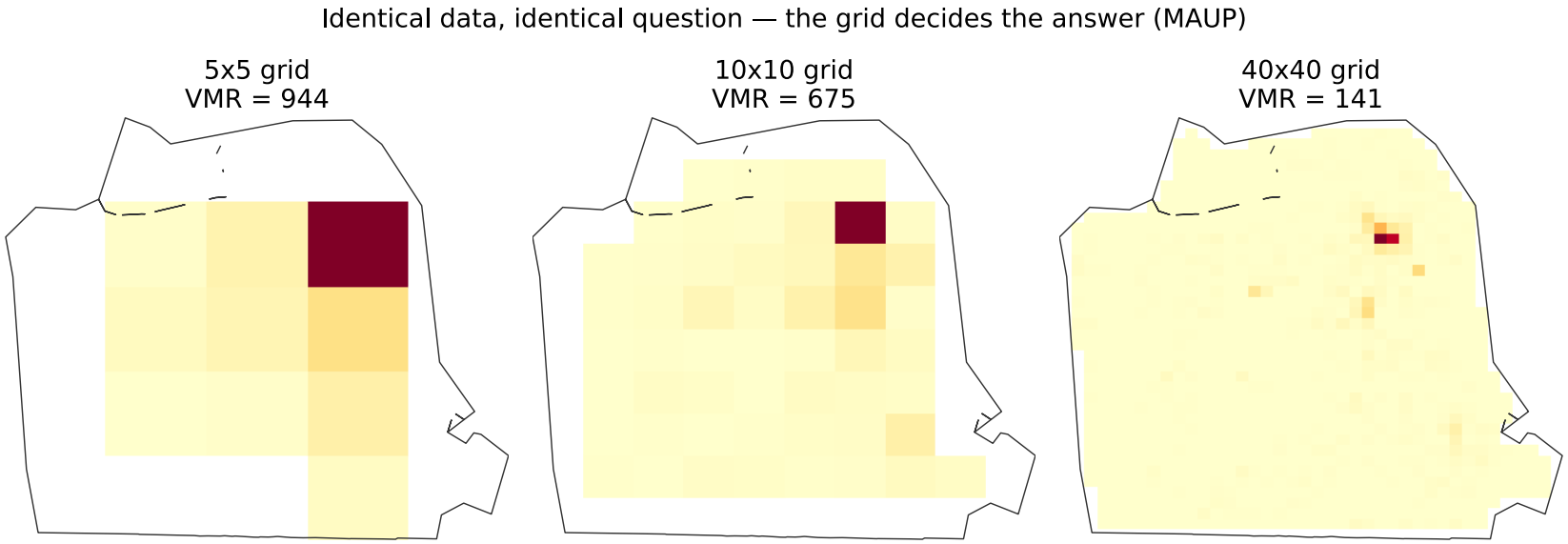
$$\chi^2 = \frac{\sum_l (n_l - \bar{n})^2}{\bar{n}} = (m-1) \cdot \mathrm{VMR} \;\sim\; \chi^2_{m-1}$$

- Large χ^2 (small p) → **reject CSR**: the pattern is genuinely non-random.
- Then shade $\hat{\lambda} = \mathrm{count}/\mathrm{area}$ per cell → a crude **hotspot map**.

675 vs an expected 1.0 — not a close call. But **car theft gives VMR = 30, also $p < 0.001$**: both "reject CSR", one is 20× the other.

> **Lesson —** confirm quadrat clustering with a χ^2 test — but report the **size** of the departure: with thousands of events, everything is significant.

MAUP — the grid is a choice, not a given



Identical data. Three grids. Three answers. VMR runs 944 → 675 → 141 as cells shrink: big cells swallow the clusters; tiny cells hold mostly 0s and 1s. Nothing about the city changed — only the grid you drew on it.

- **Mitigate:** try several resolutions; shift the grid origin.
- **Escape: KDE** and **Ripley's** K (no boxes at all).

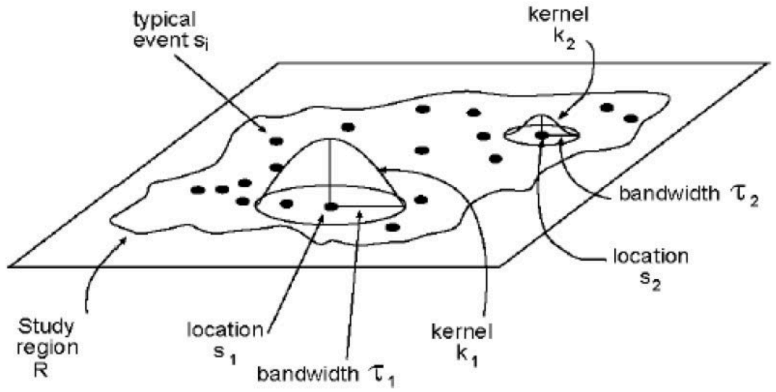
> **Lesson —** any statistic computed on an arbitrary grid inherits the arbitrariness — if the verdict changes with cell size, the cell size *is* your finding.

Kernel Density Estimation — the idea

Quadrats chop space into hard boxes; KDE gives a **smooth** surface. Drop a **bump (kernel)** on every event and **add the bumps** — where points crowd, they pile into a peak.

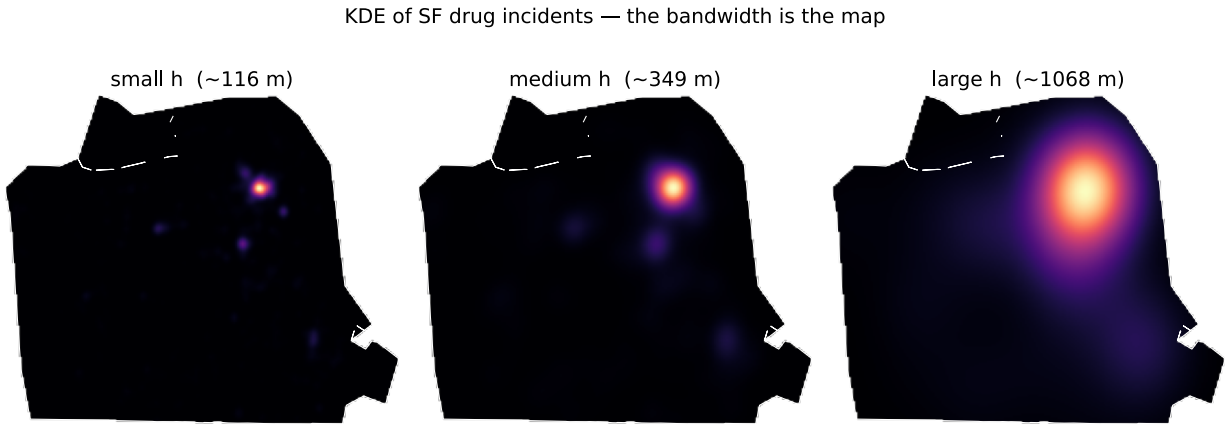
$$\hat{\lambda}(s) = \frac{1}{h^2} \sum_{i=1}^{n} K\left(\frac{\|s - s_i\|}{h}\right)$$

- K = **kernel** — a symmetric bump (Gaussian, quartic); nearer points count more.
- h = **bandwidth** — how wide each bump spreads.



> **Lesson —** KDE turns discrete dots into a continuous intensity surface by dropping a kernel on each point and summing — a grid-free upgrade to quadrats.

KDE — bandwidth is everything



The kernel *shape* barely matters; the **bandwidth** h is the map — a **bias–variance dial**:

- **Small (~116 m)** → the Tenderloin **plus** single-incident specks *(high variance: hotspots that are really noise)*.
- **Medium (~349 m)** → three credible hotspots: Tenderloin, Mission, Bayview.
- **Large (~1 km)** → one blob over half the city: true, useless *(high bias)*.

KDE — adaptive bandwidth, and what KDE cannot do

- **Adaptive** h → wider where sparse, narrower where dense: detail where you have data, stability where you don't.
- **Caveat** — KDE shows *intensity only*; it cannot tell interaction from a crowd. A dense patch may be events attracting each other, or simply more people to victimise. Separating those needs the second-order tools that follow.

> **Lesson —** too small invents hotspots, too large hides them: the bandwidth, not the kernel, makes or breaks the map.

Second-order methods — do points interact?

First-order methods map **where** events are dense. They cannot say **why**: is it a busy region, or do events actively **attract** (contagion) or **repel** (competition)?
Second-order methods measure the distances **between events** and compare them to CSR.
NNI — the quick check: mean observed nearest-neighbour distance ÷ expected under CSR, where the CSR expectation for intensity $\lambda = n/|A|$ is

$$\bar{d}_{\text{exp}} = \frac{1}{2\sqrt{\lambda}} \qquad \text{NNI} = \frac{\bar{d}_{\text{obs}}}{\bar{d}_{\text{exp}}}$$

Note what $\bar{d}_{\text{exp}}$ depends on: $|A|$, **the window you drew**. Enlarge the study area, λ falls, and the same events become "more clustered" — for free.

> **Lesson —** second-order asks whether events interact; but its yardstick is the window, so redrawing the boundary silently rewrites the answer.

Before any distance statistic: look at your coordinates

Run NNI on the SF drug incidents as downloaded: **NNI = 0.061**, "extreme clustering!"
It is an artifact. Check the coordinates first — 3,896 incidents sit at just **1,076 distinct locations (28%)**, and one coordinate holds **105** incidents.
Crime data is geocoded to **block faces and intersections**, not doorways. Thousands of pairs sit at **distance exactly 0**, so $\bar{d}_{\text{obs}}$ collapses: NNI measures **the geocoder, not the city**.

The geocoding artifact, across crime types

Deduplicate to distinct **sites** and the "extreme clustering" largely evaporates — the more tied the coordinates, the bigger the lie:

	drugs	robbery	car theft	vandalism
unique sites	28%	57%	83%	81%
NNI raw	0.061	0.346	0.613	0.580
NNI on sites	0.586	0.655	0.783	0.734

Drugs — the most tied (28% unique) — move furthest: **0.061 → 0.586**. Car theft, geocoded far more precisely (83%), barely shifts.

> **Lesson —** tied coordinates masquerade as clustering; inspect the resolution of your geocoding before any nearest-neighbour statistic, or you will publish the data provider's rounding rule as a scientific finding.

The distance functions: G, F, K

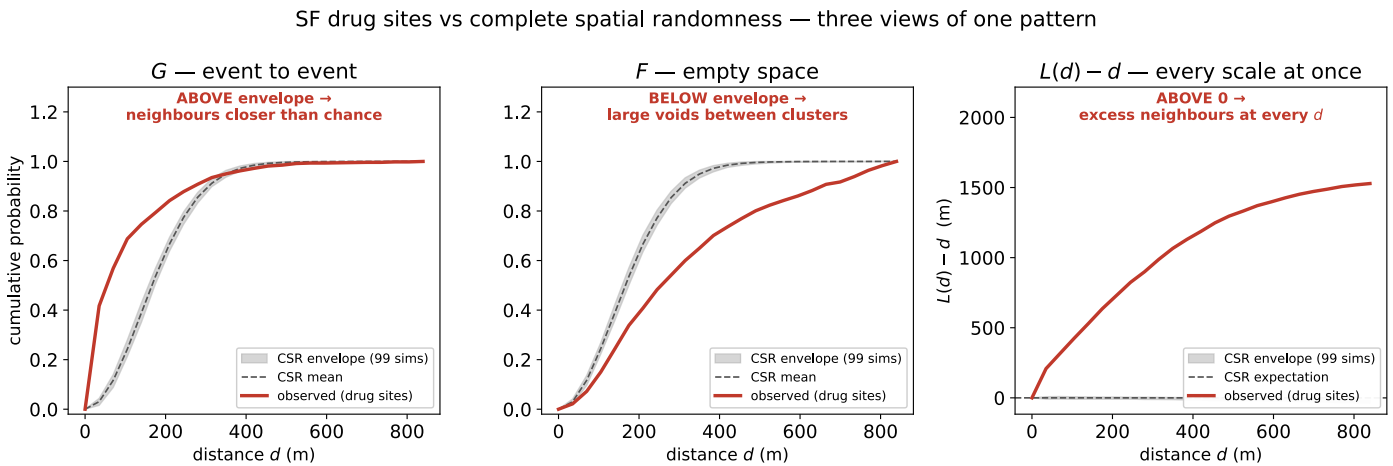
NNI uses each event's *single* nearest neighbour — one number, one scale. G, F, K use the **whole distribution**, at **every** scale.

- G **— event → event:** CDF of nearest-neighbour distances. Clustering puts neighbours close → CDF rises **early** → **above** CSR.
- F **— empty space:** CDF from *random test locations* to the nearest event. Clustering leaves **big voids** → a random spot is **far** from any event → **below** CSR.
- K **(Ripley's):** events within radius d of a typical event, ÷ λ. Above CSR → clustering. Stabilised as $L(d) - d$, so **CSR is a flat line at 0**.

The direction flips because F measures gaps, not neighbours — G up and F down say the *same* thing. Don't memorise the table; memorise *what each one probes*.

> **Lesson —** G/F read neighbour structure, K/L sweep every scale — read them together against CSR envelopes to tell clustering from repulsion.

Reading the curves — SF drug sites



The grey band is the **CSR envelope**: 99 simulated random patterns, same window, 2.5th–97.5th percentile. Outside it → not something CSR plausibly produces.

Reading the curves — what G, F and L each say

All three point the same way — because all three are outside the envelope, in the direction clustering predicts:

- G **above** — 70% of sites have a neighbour within ~120 m; CSR says ~30%.
- F **below** — random spots sit *far* from any drug site: the voids are real.
- $L(d) - d$ **above 0 at every scale** — concentration all the way out.

Remember G **up** and F **down** are the *same* verdict: F probes gaps, not neighbours.

> **Lesson —** report a distance function with its envelope, never alone: the curve only means something against the spread randomness would have produced.

Effect size: both reject CSR, one is nothing like the other

Neighbours within ~500 m of a typical site, observed vs CSR expectation:

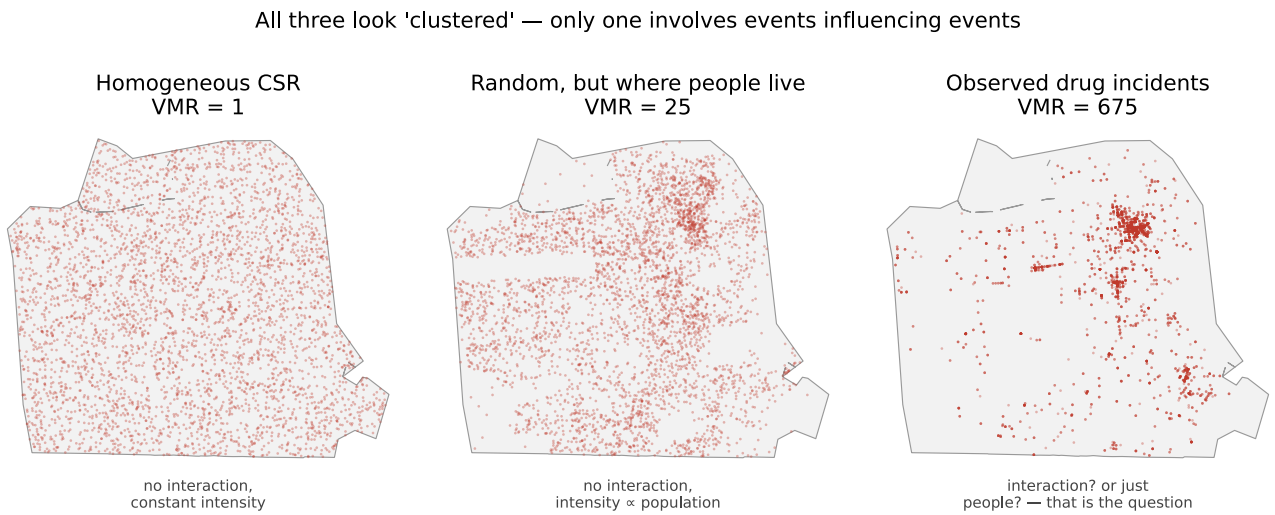
	drug sites	**car theft sites**
observed neighbours	**86.6**	36.2
expected under CSR	6.5	18.5
ratio	**× 13.4**	**× 2.0**

Both are "significant, $p < 0.001$". But drugs have **13× the neighbours** randomness predicts, car theft **twice**. Drug dealing concentrates on street corners that work; cars get stolen roughly wherever cars are parked.

And now the uncomfortable question: is car theft's ×2.0 evidence that thefts *interact*, or just that cars and people are unevenly spread?

> **Lesson —** a p-value says a departure exists; only an effect size says whether it matters — with thousands of events, the test rejects long before the pattern is interesting.

The trap: interaction, or just people?



The middle panel has **zero interaction by construction** — points scattered at random, merely **proportional to block population**. Yet VMR climbs **1 → 25** and Ripley's K calls it **clustered at every scale**: **×1.8** (10 replicates, range **1.6–2.0**).
Car theft's observed ×2.0 sits inside that range — its "clustering" is consistent with *nothing but where people are*. Drugs' ×13.4 is far beyond it. **CSR failed because intensity varies, not because events interact.**

> **Lesson —** rejecting CSR proves only that the homogeneous-Poisson story is wrong; blaming interaction when the cause is uneven intensity is the field's most common error.

The fix: pick the null that matches your question

This is **Part 1's three suspects**, in point-pattern clothing:

Suspect	Point-pattern name	The fix
Shared cause (confounding)	**inhomogeneous** intensity $\lambda(s)$	put the driver *in the null*
Contagion (diffusion)	genuine **interaction**	what's left after you do

- **Inhomogeneous Poisson null** — simulate with $\lambda(s)$ estimated from a covariate (population, land use, roads) instead of a constant. Clustering that survives *that* envelope is interaction.
- K_{inhom} — Ripley's K with the varying intensity divided out.
- **Case–control** — compare cases against a *control* pattern carrying the same population-at-risk (disease mapping's standard move).

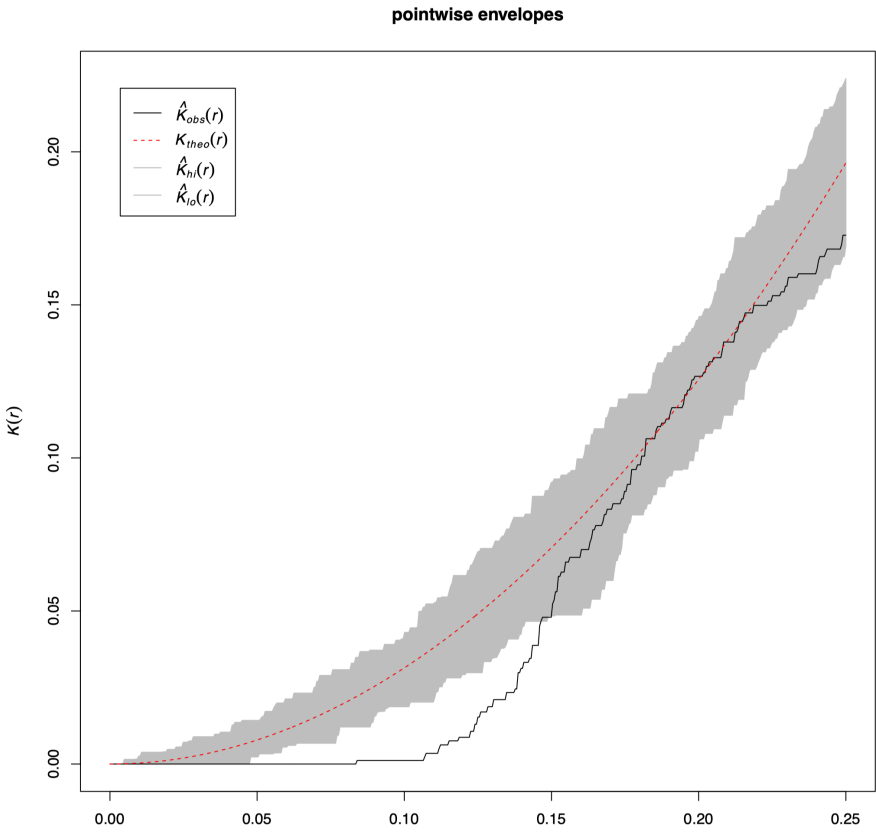
Same map, three stories — the null is where you encode which story you are testing.

> **Lesson —** the interesting choice in point pattern analysis is not the statistic but the null: change what randomness means and the same data change their verdict.

Point patterns — practical notes (1/2): the data and the window

Cross-cutting cautions for G, F, and K alike — the three that bite hardest come *before* you compute anything:

- **Edge correction** — boundary events have unobserved neighbours *outside* the window, biasing every distance statistic downward.
- **The window is an assumption** — NNI, λ and every envelope depend on it. Use the real study area, never a bounding box with ocean in it.
- **Tied coordinates** — check **unique(coords)** before trusting any NN statistic.



Point patterns — practical notes (2/2): the null and the tools

- **Envelopes, not eyeballs** — Monte-Carlo with the null you actually mean: homogeneous CSR
  answers a different question from an inhomogeneous $\lambda(s)$.
- **Effect size, not just p** — with thousands of events the test rejects long before the pattern is interesting.
- **Implementations** — `pointpats` (`g_test`, `f_test`, `k_test`); R/ `spatstat` is canonical.

> **Notebook:** `notebooks/04b_point_patterns.ipynb` — the SF analysis end-to-end.

Point patterns — takeaways

1. **The locations are the data**; the window is part of the data.
2. **Everything is measured against a null.** CSR = constant intensity **+** independence.
3. **First-order** (quadrats, KDE) maps *where*; **second-order** (G, F, K) asks *whether events interact*.
4. **Every first-order tool has a scale knob** — quadrat size, bandwidth — and the knob changes the answer (MAUP).
5. **Look at your coordinates first**: SF drugs, NNI 0.061 → 0.586 after de-duplication.
6. **Report effect size**, not just p: ×13.4 (drugs) vs ×2.0 (car theft).
7. **Rejecting CSR is ambiguous** — population inhomogeneity alone gives ×1.8 (1.6–2.0) with zero interaction, bracketing car theft's ×2.0. Choose the null that fits the question.

> **Lesson —** point pattern analysis is null-model design; the statistics are easy, and deciding what "random" should have meant is the entire scientific contribution.

Part 3: Spatial Dependence Strategies

Three Main Approaches:

1. **Feature Engineering**: Add spatial context to input variables
2. **Model Structure**: Embed dependence in model equations/assumptions
3. **Regularization**: Add spatial smoothness penalties to objective function

Choice Criteria:

- **Data type**
- **Computational resources**
- **Interpretability needs**
- **Uncertainty quantification**

Approach 1: Spatial Feature Engineering

Definition: Transform spatial relationships into input **features** for standard ML/statistical models
Core Idea: Make spatial dependence **explicit** in the feature space

Common Approaches:

- **Spatial lags**: Include neighbors' values as features
- **Kernel features**: Gaussian/exponential weighted neighborhoods
- **Spatial aggregations**: Local averages, densities, gradients
- **Distance features**: Distances to landmarks, boundaries, centroids
- **Graph-derived embeddings**: learn compact node representations (via node2vec/DeepWalk or GNN encoders) that capture multiscale connectivity and local structure
- **Texture features**: Local variance, local Moran's I, edge density

> **Notebook:** **notebooks/03b_features_engineering.ipynb** — coordinates, spatial lags, MSEF, and GCN embeddings scored by residual Moran's I.

Moran Spatial Eigenvector Filtering (MSEF)

- **Definition:** Uses **eigenvectors** derived from a (centered) spatial weights matrix to represent principal spatial patterns (from broad to fine scales). These eigenvectors capture orthogonal spatial structures and can be included as additional regressors to model spatial variation or remove residual spatial autocorrelation.

- **How it works (brief):**
  i. Build a spatial weights matrix W encoding the neighborhood (contiguity, KNN, kernel, or flow-based).
  ii. Center W using the mean-centering matrix $C = I - \frac{1}{n}\mathbf{1}\mathbf{1}^\top$, then compute $B = CWC$.
  iii. Compute eigenpairs of B: $Bv_k = \lambda_k v_k$. Eigenvectors v_k with large positive eigenvalues λ_k reflect positive spatial autocorrelation at different scales.
  iv. Select a **subset of eigenvectors** (see selection criteria) and include them as additional regressors or features in your model.

When to use:

- As a flexible, model-agnostic way to add spatial features to ML pipelines.
- To orthogonalize spatial patterns so that standard regression/ML methods can handle spatial structure without changing their core estimators.

Selection tips:

- **Rank eigenvectors by their associated Moran's I** — free, not a second pass: each v_k with $\lambda_k \neq 0$ is already centered, so $I(v_k) = \frac{n}{S_0}\lambda_k$. **Moran order *is* eigenvalue order**; the rescaling only makes λ_k readable, and $\frac{n}{S_0}\lambda_{\min}$, $\frac{n}{S_0}\lambda_{\max}$ are the **Moran bounds** — the true range of I, not $[-1, 1]$.
- **Threshold**: keep candidates with $I(v_k)/I_{\max} = \lambda_k/\lambda_{\max} > 0.25$ (Griffith's rule).
- **Use forward selection** (AIC/BIC/CV) to avoid **overfitting** and **multicollinearity**.
- **Limit the number of eigenvectors** (e.g., tens, not hundreds) based on sample size and parsimony.

Caveats:

- **Symmetry is required**: $B = CWC$ inherits any asymmetry of W, so a **row-standardized** W yields complex eigenvalues — and **numpy.linalg.eigh** reads only one triangle, returning wrong eigenvectors *without error*. Use binary contiguity, or symmetrize via $(W + W^\top)/2$.
- Results depend on the choice of W; always run **sensitivity checks** across plausible W.
- Selected eigenvectors can be correlated with covariates — check multicollinearity and interpret coefficients cautiously.
- Eigenvectors capture spatial structure but do not identify causal mechanisms.

References: Griffith (2003) on ESF; Dray et al. (2006) for eigenfunction methods.

Feature Engineering — Advantages and Limitations

Pros:

- **Model agnostic**: Works with any ML algorithm
- **Computational efficiency**: Standard optimization, no special software
- **Easy interpretation**: Features have clear spatial meaning
- **Flexible**: Can combine multiple spatial representations

When to Use:

- Rapid **prototyping** and benchmarking
- Large-scale prediction systems
- When spatial structure is well-understood
- **Integration** with existing ML pipelines

Cons:

- **Manual engineering**: Requires domain knowledge and experimentation
- **Feature proliferation**: May create high-dimensional, correlated features
- **Static neighborhoods**: Fixed spatial relationships, not adaptive
- **No uncertainty quantification**: Point estimates only

Strategy 2: Spatial Model Structure

Definition: Incorporate spatial dependence directly into model equations/assumptions
Core Idea: **Model the process generating spatial dependence** rather than just adding spatial features

Main model families:

1. **Spatial econometrics models** : **SLX, SAR, SEM, SDM, SDEM, SAC**
2. **Kriging** : Gaussian Process-based optimal prediction for point-reference data
3. **Hierarchical Models** : Multi-level spatial frameworks combining structured and unstructured effects

Spatial Econometric Models

Concepts first: what is “spatial” in these models?

- Two mechanisms
  - **Spillover in outcomes** (endogenous): neighboring outcomes interact $\longrightarrow$ global feedback possible
  - **Spillover in covariates** (exogenous): neighboring X influence local y
- Two spatial scales
  - **Local**: effects fall only on immediate neighbors
  - **Global**: effects propagate to neighbors-of-neighbors (**feedback loops**)

Model mapping (quick mental model)

- **SLX**: local exogenous spillovers
- **SAR**: global endogenous spillovers
- **SEM**: global error diffusion (correlated residuals)
- **SDM**: SAR + SLX (global endogenous + local exogenous)
- **SDEM**: SEM + SLX (global diffusion + local exogenous)
- **SAC/SARAR**: SAR + SEM (both endogenous and error diffusion)

— choose based on theory and diagnostics, not only fit.

SLX — Spatial Lag of X (local exogenous spillovers)

Equation:

$$Y = X\beta + WX\,\theta + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Interpretation:

- Coefficients in θ measure how neighbors' characteristics affect Y_i (local spillovers)
- Linear model $\longrightarrow$ coefficients have usual interpretation (no feedback multiplier)

When to use:

- **Expect neighborhood attributes to matter, but not feedback** in Y
- Clean, interpretable spillovers needed; **baseline** for design-based work

Pros/Cons:

- Pros: OLS-estimable, transparent interpretation; great starting point
- Cons: No endogenous feedback; ignores global propagation

SAR — Spatial Autoregressive (Lag) Model

Equation:

$$Y = \rho WY + X\beta + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Interpretation:

- Endogenous interaction/feedback among outcomes
- **Spillover** effects via $(I - \rho W)^{-1}$ (**spatial multiplier**)
- **Direct** and **indirect** impact decomposition

When to Use:

- Peer effects, diffusion processes
- Network spillovers in outcome variable
- Policy impact analysis with spatial feedback

SEM — Spatial Error Model

Equations:

$$Y = X\beta + u, \quad u = \lambda W u + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Interpretation:

- Captures **omitted spatially structured shocks**
- Corrects inference when residuals cluster spatially
- Focus on consistent β estimation

When to Use:

- **Spatial correlation due to unobserved variables**
- Model **misspecification** creates spatial residual patterns
- Primary interest in covariate effects, not spatial spillovers

SDM — Spatial Durbin Model

Equation:

$$Y = \rho WY + X\beta + WX\theta + \varepsilon$$

Interpretation:

- Allows spillovers through both outcomes and covariates
- WX captures exogenous neighbor effects
- Nests SAR/SEM under parameter restrictions

When to Use:

- Suspect both endogenous and exogenous spillovers
- Policy diffusion analysis
- Robust to some W misspecification

Advantage: Detailed direct/indirect/total impact decomposition

SDEM — Spatial Durbin Error

Equations:

$$Y = X\beta + WX\,\theta + u, \quad u = \lambda W u + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$

Interpretation:

- Local exogenous spillovers (via WX) + global error diffusion (via λ)
- Still linear in $Y \rightarrow$ coefficients in β, θ interpretable as usual

When to use:

- No theoretical need for endogenous feedback in Y, but residual diffusion likely
- Prefer linear interpretation of coefficients without impact multipliers

Pros/Cons:

- Pros: Interpretable parameters; efficient vs SLX when errors are spatial
- Cons: Misspecified if true process has endogenous feedback (favor SDM then)

SAC / SARAR — Spatial combo (lag + error)

Equations:

$$Y = \rho WY + X\beta + u, \quad u = \lambda Wu + \varepsilon$$

Interpretation:

- Endogenous outcome feedback and spatially correlated errors simultaneously

Notes:

- Can be useful when residual autocorrelation remains after SAR/SDM
- Identification and interpretation can be delicate; avoid fully unrestricted “Manski” models

Explanation and Interpretation of Parameters ρ, θ, and ε

ρ (rho):

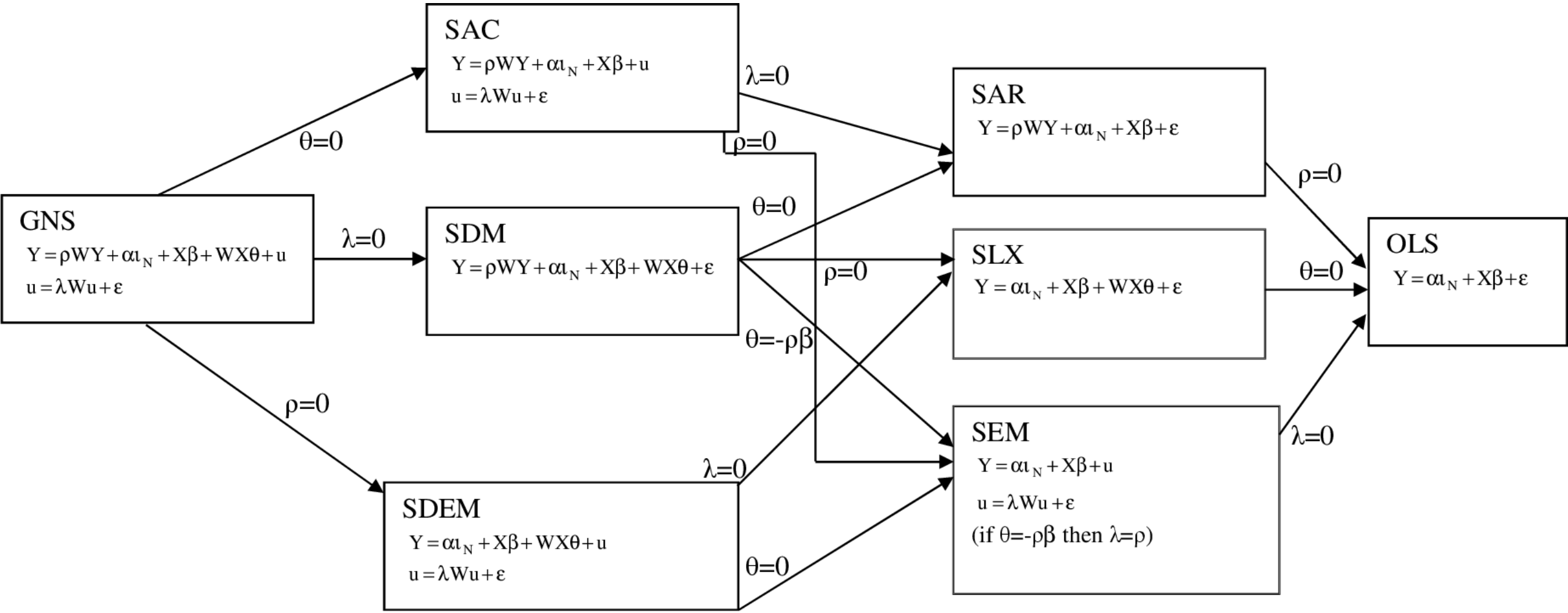
- Represents the strength of endogenous spatial dependence (feedback) in outcome variable Y.
- In SAR/SDM/SAC models, ρ quantifies how much neighboring outcomes (WY) influence each location.
- **Interpretation:**
  - $\rho = 0$: No spatial feedback; outcomes are independent.
  - $0 < \rho < 1$: Positive spatial spillovers; high values in neighbors increase local Y.
  - $\rho < 0$: Negative spatial spillovers (rare); high values in neighbors decrease local Y.
- **Magnitude:** Larger $|\rho|$ means stronger spatial autocorrelation and more global propagation of effects.

θ (theta):

- Measures exogenous spatial spillovers from neighbors' covariates (WX).
- In SLX/SDM/SDEM models, θ captures how neighbors' characteristics affect local outcomes.
- **Interpretation:**
  - $\theta_j > 0$: Higher values of covariate X_j in neighbors increase local Y.
  - $\theta_j < 0$: Higher values of X_j in neighbors decrease local Y.
- **Magnitude:** Indicates the average effect of neighboring covariates, controlling for local X.

ε (epsilon):

- Represents random error or noise, assumed to be normally distributed and independent across locations.
- In SEM/SDEM/SAC models, ε is the innovation term after accounting for spatial structure.
- **Interpretation:**
  - Captures unexplained variation not modeled by X, WX, or spatial feedback.
  - If residuals show spatial autocorrelation, model may need SEM/SAC structure.


GNS
$Y = \rho WY + \alpha\iota_N + X\beta + WX\theta + u$
$u = \lambda Wu + \varepsilon$
SAC
$Y = \rho WY + \alpha\iota_N + X\beta + u$
$u = \lambda Wu + \varepsilon$
SDM
$Y = \rho WY + \alpha\iota_N + X\beta + WX\theta + \varepsilon$
SDEM
$Y = \alpha\iota_N + X\beta + WX\theta + u$
$u = \lambda Wu + \varepsilon$
SAR
$Y = \rho WY + \alpha\iota_N + X\beta + \varepsilon$
SLX
$Y = \alpha\iota_N + X\beta + WX\theta + \varepsilon$
SEM
$Y = \alpha\iota_N + X\beta + u$
$u = \lambda Wu + \varepsilon$
(if $\theta = -\rho\beta$ then $\lambda = \rho$)
OLS
$Y = \alpha\iota_N + X\beta + \varepsilon$
θ=0
λ=0
ρ=0
θ=0
λ=0
ρ=0
θ=-ρβ
ρ=0
λ=0
θ=0
ρ=0
θ=0
λ=0

Picking a model: diagnostics and strategy

Don't pick a spatial model by taste — let the OLS residuals tell you which kind of dependence is left. The Lagrange-Multiplier tests point you toward a lag or an error specification, and their robust versions arbitrate when both fire.

- **Start from OLS** $\longrightarrow$ **run spatial diagnostics on residuals**
  - Moran's I on residuals; LM lag and LM error; robust LM versions; SARMA joint test
- **Specific-to-general search (often recommended)**
  - If LM-error wins $\longrightarrow$ try SEM/SDEM
  - If LM-lag wins $\longrightarrow$ try SAR/SDM
  - If both significant $\longrightarrow$ prefer Durbin specs (SDM/SDEM) and test down
- **General-to-specific complement**
  - Start from SDM or SAC; test common-factor restrictions to reduce to SAR/SLX
- **Always re-check residual spatial autocorrelation after fitting**

Interpreting coefficients and impacts

Once an outcome lag ρWY enters the model, the raw coefficients are no longer the effects: a change in one place ripples through neighbours and loops back. We therefore report the effect as a **direct** piece (own area), an **indirect** piece (spillover to others), and their **total**.

- Models with ρWY (SAR, SDM, SAC) are **non-linear** in Y
  - Coefficients β are not marginal effects
  - Report **direct**, **indirect**, and **total impacts** from $(I - \rho W)^{-1}$
- SLX/SDEM
  - θ on WX are interpretable as average neighbor effects (no multiplier)

> Practice: Always report impacts (and their uncertainty) when $\rho \neq 0$

Estimation choices (software perspective)

The same spatial model can be fit three ways — maximum likelihood, instrumental-variables/2SLS, or GMM — trading statistical efficiency for robustness. The worked example next takes the IV route (**GM_Lag**); here is the menu.

- Estimator families (all in **spreg**)
  - **SLX**: plain OLS (no endogenous lag to instrument)
  - **ML** (**ML_Lag**, **ML_Error**): efficient under normality — the classic SAR/SEM likelihood
  - **IV / 2SLS** (**GM_Lag**): instruments WY with spatial lags of X; robust SEs and impacts available
  - **GMM** (**GMM_Error**): moment-based SEM with robust/heteroskedastic variants

Estimation choices — instruments & robustness

- Instruments and higher-order lags
  - Lag models instrument WY using spatial lags of X by default
  - With WX on the RHS (e.g., SDM), use higher-order instruments (set **w_lags** ≥ 2) or provide your own
- Robustness
  - Use heteroskedasticity-robust variance if residual tests suggest heteroskedasticity
- Reporting
  - Always report the estimator, the **W** /Graph definition, and which diagnostics drove the model choice

Worked example — bars and drunk incidents in SF (1/3)

Do bars raise a neighbourhood's drunk-incident rate — and how much lands **next door**? **580 San Francisco block groups**: build a Queen graph, *audit it*, then test the outcome for clustering.

```
import geopandas as gpd, esda
from libpysal.graph import Graph

g  = Graph.build_contiguity(gdf, rook=False)     # Queen contiguity
mi = esda.Moran(gdf["DrunkP1k"], g.to_W(), permutations=999)
```

- **Audit first**: drop the Farallon Islands (**0** neighbours), Golden Gate Park (**35** — a hub), and a multipart island set → **577** block groups, one component.
- Global Moran's I **= 0.384**, **p_sim = 0.001** → **incidents cluster** — a cue to check the residuals, not yet a licence to model space.

> **Notebook:** **notebooks/03_spatial_econonometric_modeling.ipynb** — this example end to end, step by step: graph audit → weights → diagnostics → the family by AIC → impacts → elasticities.

Worked example — OLS diagnostics (2/3)

Bridge the **Graph** to the row-standardised **W** that **spreg** expects, then fit OLS with spatial diagnostics. The **residual** read-out decides the model.

```
import spreg
w = g.to_W(); w.transform = "r"          # row-standardised W
y = gdf[["DrunkP1k"]].values
X = gdf[["pHHPov", "BarPSqMi", "RetailPSqMi",
         "pMale", "VacantHU"]].values

ols = spreg.OLS(y, X, w=w, spat_diag=True, moran=True)
```

Diagnostic	value	p
Moran's I (residuals)	**0.222**	0.0000
LM-lag / robust LM-lag	102.1 / **16.93**	0.0000 / 0.0000
LM-error / robust LM-error	86.2 / **1.04**	0.0000 / **0.31**

OLS diagnostics — reading the tests

Reading: residual Moran's I is **0.222** (**p < 0.0001**) — the covariates did *not* absorb the spatial pattern, so **OLS is rejected**: its standard errors are wrong and, with a lag present, its coefficients are biased too.
Which term is missing? Both plain LM tests fire, so they cannot discriminate — that is what the **robust** pair is for, each controlling for the other:

- robust **LM-lag** **= 16.93** (**p < 0.0001**) → **significant**
- robust **LM-error** **= 1.04** (**p = 0.31**) → not significant

> **Lesson —** when both LM tests fire, let the *robust* pair name the culprit: here the data want the **lag** (ρWY), not the error.

Worked example — spatial lag impacts (3/3)

With $\rho \approx 0.47$ the multiplier $(I - \rho W)^{-1}$ splits each β into **direct** and **indirect** — `spreg.GM_Lag(..., spat_impacts="full")`. Use `"full"`, not `"simple"`: `"simple"` pins the direct multiplier at 1.0, misfiling the feedback that **loops back**.

BarPSqMi	Direct	Indirect	Total
impact	0.025	0.020	**0.045**

- **10 more bars/sq mi → +0.45** incidents/1,000 — **44% lands outside** the block group that gained them. Elasticity: **+10% bars ⇒ +1.9%**.
- **It worked**: residual Moran's I **0.222 → 0.003** (**p ≈ 0.39**).

> **Lesson —** report **impacts, not** β: OLS said 0.032, the lag model 0.024 — the spillover is what the licensing neighbourhood never pays for.

Quick decision cheat sheet (advanced)

- Start: OLS + diagnostics (Moran's I, LM/Robust LM, Durbin tests)
- Robust LM-Error only $\longrightarrow$ SEM (or SDEM if SLX plausible)
- Robust LM-Lag only $\longrightarrow$ SAR (or SDM if SLX plausible)
- Both robust tests significant $\longrightarrow$ SDM; test down via common-factor restrictions
- AK significant in 2SLS $\longrightarrow$ spatial structure remains; consider SLX/SDM/SDEM with valid instruments

References: spreg "Specification tests" notebook; Koley & Bera (2024); Anselin & Rey (2014).

> **Practical Example**
> More worked examples (hedonics)

Best practices for applied work (checklist)

- **Theory first**: decide whether endogenous feedback is plausible
- Weights W: **try multiple reasonable graphs** (contiguity, KNN, distance) and **row-standardize**
- **Diagnostics**: LM tests, robust variants, and residual Moran's I
- **Prefer Durbin models** (SDM/SDEM) **when in doubt about exogenous spillovers**
- **Impacts**: compute and report direct/indirect/total effects for lag models
- **Sensitivity**: test conclusions across alternative W and sample definitions
- **Validation**: use spatial cross-validation; check for remaining spatial structure in residuals
    - After fitting: residual Moran's I should not be significant; otherwise revise W/specification

Conditional Autoregressive (CAR) — the other tradition

For **areal** data, two traditions: the econometric family (SAR/SEM/SDM) puts space in the **mean** (spillovers); **CAR** puts it in a **latent random effect** ϕ (smoothing).

- **Idea:** fit `Y ~ X`; if residuals stay autocorrelated, add a spatial random effect ϕ whose CAR prior lets each area **borrow strength from its neighbours**.
- **Home turf:** Bayesian hierarchical models, count/rate outcomes, **disease mapping** (BYM).

	Econometric (SAR/SEM/SDM)	CAR
Space enters	the mean (WY, Wu, WX)	a latent effect ϕ (prior)
Question	spillovers / impacts	smoothing / borrowing strength
Estimation	ML · GMM / 2SLS	Bayesian (MCMC · INLA)

> **Lesson —** same areal dependence, two questions: reach for **SAR/SDM** to *measure spillovers*, **CAR** to *smooth a noisy latent field* (e.g. disease risk).

Kriging for Point-Reference Data

Concept: **Interpolate a spatial process at unsampled locations** using a covariance (or semivariogram) model.

Kriging — Quick Overview

- Prediction at an unsampled location as a **weighted average** of nearby observations.
- The weights are chosen to be **optimal** so the predictor is **unbiased** and has **minimum variance** given the assumed covariance/variogram model.
- Outputs: **a point prediction** (the kriged value) and a **per-location uncertainty measure** (the kriging variance).

Ordinary Kriging — prediction system

Objective: predict Z at location s0 as a linear unbiased estimator with minimum variance.
Prediction (linear estimator)

$$\hat{Z}(s_0) = \sum_{i=1}^{n} \lambda_i Z(s_i)$$

Unbiasedness constraint

$$\sum_{i=1}^{n} \lambda_i = 1$$

- How to interpret the kriging weights and prediction
  - **Weights reflect the spatial configuration and the fitted variogram/covariance**: closer and more informative neighbors get larger weights; distant or noisy neighbors get low weights.
  - **The prediction is not a simple smoothing average**: it accounts for spatial correlation structure (range, sill, nugget) so it adapts to the effective spatial horizon.

- How to interpret the kriging variance
  - **The kriging variance is the model-based estimate of uncertainty for the predicted value at that location** (smaller where observations are dense and correlations are strong; larger in sparse areas or beyond the range).
  - It is a function of the variogram/covariance and sampling geometry only — **it does not depend on the observed values** themselves (only on their **locations** and the fitted model).
  - Use it to build approximate prediction intervals (under normality or large-sample approximations) and to prioritize further sampling (**high variance → useful new sample**).

Practical Workflow

1. **Compute empirical semivariogram** (choose distance bins).
2. **Fit** theoretical model (spherical, exponential, Gaussian, Matérn).
3. Build covariance/variogram matrix and **solve** kriging system $\rightarrow$ **weights** $\rightarrow$ **predictions**.
4. Optionally **detrend** or include trend terms (universal kriging).
5. **Validate** with spatial CV (blocked CV) or leave-one-out.

Outputs & Validation

- Outputs: predicted value and kriging variance at each target location.
- Validation: spatial CV, LOO errors, and check residual variogram (should be near pure nugget).

Strengths

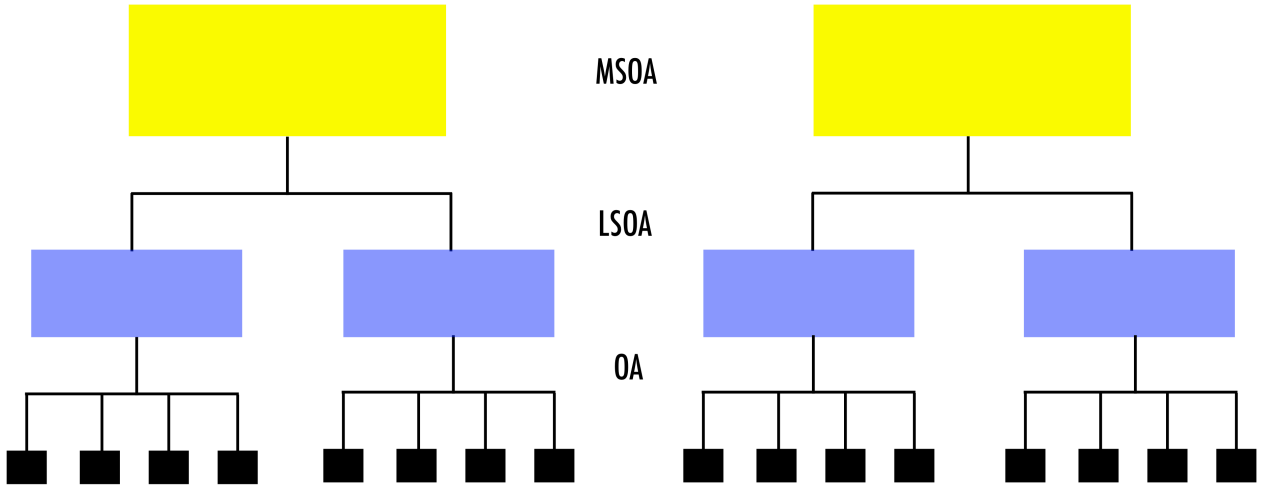
- Optimal linear interpolation under model assumptions.
- Explicit **uncertainty quantification** (kriging variance).
- **Flexible**: supports trends and auxiliary variables (cokriging / regression kriging).

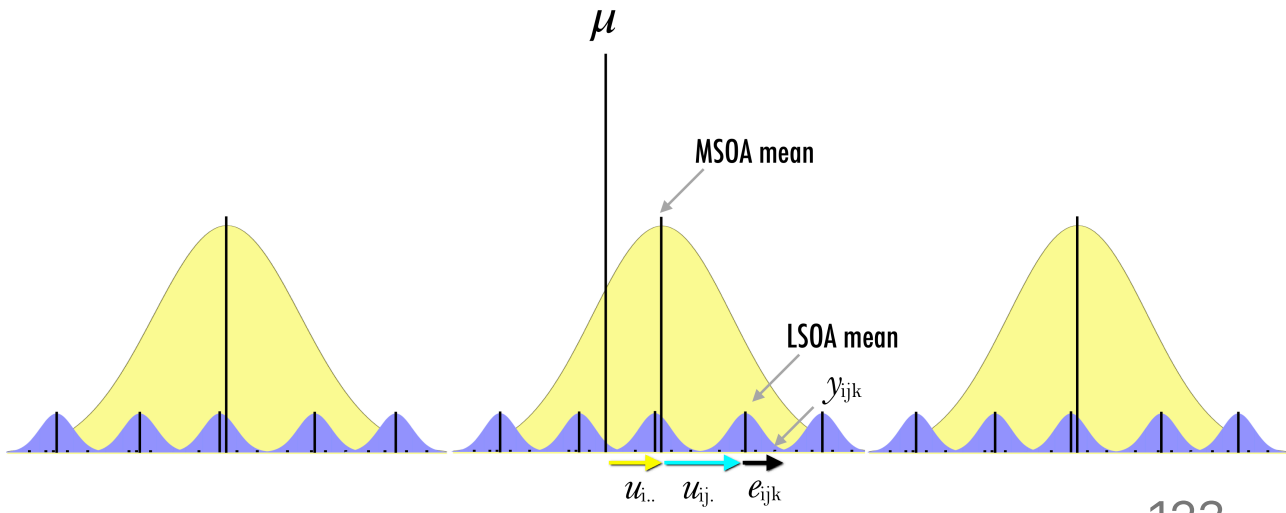
Limitations & Scalability

- **Sensitive** to variogram choice and fitting.
- Assumes **stationarity** (or **local stationarity**); detrend if needed.
- **Computationally heavy** for large n.

Note: Hierarchical Spatial Models (optional)

- **Multi-level** / **mixed-effects** frameworks to model grouped data with spatial dependence (e.g., CAR/ICAR for areal data; GP/SPDE for point-referenced).
- Out of scope for today; recommended readings: Multilevel Modelling - Part 1 Multilevel Modelling - Part 1





Model Structure: Recap

Advantages

- **Theoretical foundation**: Grounded in spatial process theory
- **Interpretable parameters**: Range, spillover intensity, etc.
- **Uncertainty quantification**: Full probabilistic framework
- **Scientific insight**: Direct vs indirect effects, spatial mechanisms

Limitations

- **Strong assumptions** (stationarity, parametric dependence forms)
- **Computational complexity** (dense matrix ops, factorization)
- **Model selection challenge** (many competing variants)
- **Scalability concerns**

When to Prioritize Model Structure

- Need scientific mechanism insight (spillovers, diffusion)
- Formal inference / hypothesis testing required
- Uncertainty quantification critical
- Spatial structure strong & theory-supported
- Dataset size manageable or sparse methods available

Strategy 3: Spatial Regularization

Definition: **Add spatial smoothness penalties to learning objectives**
Core Idea: Encourage **similar predictions** for **nearby locations**
Mathematical Framework:

$$\min_{\theta} \underbrace{\sum_{i} \ell(y_i, f_\theta(x_i))}_{\text{Data fit}} + \underbrace{\lambda \sum_{(i,j)\in E} w_{ij} \|\theta_i - \theta_j\|^2}_{\text{Spatial penalty}}$$

Regularization Types:

- **Parameter smoothing:** Local model parameters vary smoothly
- **Prediction smoothing:** Similar predictions for neighbors
- **Graph total variation:** Piecewise-constant spatial structure
- **Embedding regularization:** Smooth latent representations

Flexibility: Can be added to most objective functions

Regularization — Advantages and Limitations

Pros:

- **Model flexibility**: any differentiable objective
- **Efficiency**: standard optimisation + a term
- **Tunable smoothness**: via λ
- **Scales well**: linear in graph edges

Cons:

- **Hyperparameter tuning**: λ and graph
- **Over-smoothing**: may blur discontinuities
- **No uncertainty**: point estimates only
- **Graph dependence**: sensitive to the graph

When to use: complex non-parametric models, large datasets, a reasonable local-smoothness assumption, prediction valued over interpretation.

Strategy Comparison and Selection Guide

Aspect	Feature Engineering	Model Structure	Regularization
Ease of Implementation	High	Medium	Medium
Computational Cost	Low	High	Medium
Theoretical Foundation	Low	High	Medium
Uncertainty Quantification	None	Full	Limited
Scalability	Excellent	Poor	Good
Interpretability	High	High	Medium

Decision Framework:

1. **Start with feature engineering** for baseline and rapid iteration
2. **Use model structure** when interpretation and uncertainty quantification are critical
3. **Apply regularization** for complex models and large datasets
4. **Combine approaches** for best performance (hybrid methods)

Implementation Decision Framework

Step 1: Data Diagnosis

- Visualize spatial patterns and covariate relationships
- Test for spatial dependence (Moran's I, variograms) and check stationarity
- Note mechanisms to guide model choice (spillover, diffusion, shared context)

Step 2: Method Selection

- Feature engineering — fast baselines and easy integration with ML
- Model structure — SAR/SDM/SEM/CAR when theory or inference requires it
- Regularization / hierarchical models — for scale, complexity, or multilevel processes

Step 3: Implementation Strategy

- Try multiple plausible Ws for sensitivity
- Recompute graph-derived features (e.g., WX) inside CV folds when needed
- Use spatial cross-validation for performance and hyperparameter tuning
- Balance interpretability vs compute cost; document W, transforms, and seeds

Step 4: Model Validation

- Residual checks: Moran’s I and variogram of residuals (should be near zero)
- Compare approaches on spatial CV and uncertainty calibration (pred intervals)
- Sensitivity checks for W, model form, and key parameters

Python Software Ecosystem

Core Spatial Analysis Libraries

Spatial Weight Matrices & Econometrics:

- **libpysal**: Spatial weights, autocorrelation measures
- **spreg**: SAR, SEM, SDM estimation
- **pysal**: Comprehensive spatial analysis suite

Geostatistics & Kriging:

- **scikit-gstat**: Variogram modeling and kriging
- **pykrige**: Ordinary, universal, regression kriging
- **gstools**: Advanced geostatistical modeling
- **geostatspy**: Educational geostatistics workflows

Hierarchical & Bayesian Models:

- **pymc**: Bayesian spatial models with MCMC
- **stan**: Advanced hierarchical spatial modeling
- **inla**: Fast approximate Bayesian inference (R interface)